

Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
Computación Paralela y Distribuida — Semestre 2, 2026

Proyecto 1 — Entrega final

Screensaver "Túnel Caleidoscópico de Interferencia": versión
secuencial y paralelización con OpenMP

Integrantes

Carlos Daniel Estrada Vega — 20853

André Emilio Pivaral López — 23574

Daniela Ramírez de León — 23053

Guatemala, 23 de septiembre de 2026

Índice

1. Introducción
2. Antecedentes
 - 2.1 OpenMP y el modelo de memoria compartida
 - 2.2 Metodología PCAM
 - 2.3 Speedup, eficiencia y ley de Amdahl
 - 2.4 SDL 2
3. Cuerpo
 - 3.1 Descripción del screensaver
 - 3.2 Física y trigonometría del modelo
 - 3.3 Diseño con PCAM
 - 3.4 Versión secuencial (v0)
 - 3.5 Versiones paralelas (v1, v2, v3)
 - 3.6 Protección de la memoria compartida y sincronía
 - 3.7 Programación defensiva y manejo de recursos
 - 3.8 Verificación de correctitud
 - 3.9 Metodología de medición
 - 3.10 Resultados
 - 3.11 Análisis
4. Conclusiones
5. Recomendaciones
6. Referencias
7. Apéndices — Anexo 1 (diagrama de flujo), Anexo 2 (catálogo de funciones), Anexo 3 (bitácora de pruebas), capturas

1. Introducción

El objetivo de este proyecto es diseñar un programa secuencial con potencial de paralelización y luego acelerarlo con OpenMP de forma iterativa, midiendo en cada paso el speedup y la eficiencia obtenidos. El programa elegido es un *screensaver* llamado "Túnel Caleidoscópico de Interferencia": N fuentes de onda orbitan en un plano, y el color de cada píxel se obtiene de la superposición de sus N ondas, después de aplicar transformaciones polares que producen un túnel con simetría de caleidoscopio.

La elección no es casual. El color de cada píxel se calcula de forma independiente, pero cuesta $O(N)$ operaciones trigonométricas, por lo que un frame de 800×600 con $N = 64$ exige unos 30 millones de evaluaciones de onda. Es un problema intensivo en cómputo, con paralelismo de datos casi perfecto, en el que el cálculo ocurre deliberadamente en el CPU (y no en la GPU mediante *shaders*) para que haya carga que paralelizar con OpenMP.

Este informe describe el modelo físico y matemático, el diseño de la solución con la metodología PCAM, las tres versiones paralelas construidas sobre la secuencial, los mecanismos de sincronía empleados y los resultados de las pruebas de rendimiento: speedup, eficiencia, FPS alcanzados y el N máximo que se puede sostener a 60 FPS.

2. Antecedentes

2.1 OpenMP y el modelo de memoria compartida

OpenMP es una interfaz de programación para paralelismo en memoria compartida, basada en directivas de compilador, rutinas de biblioteca y variables de entorno. Su modelo de ejecución es *fork-join*: el programa inicia con un hilo, y al encontrar una región `parallel` se crea un equipo de hilos que la ejecuta; al terminar la región, los hilos se sincronizan y sigue solo el hilo principal. Según la especificación, al final de toda construcción de reparto de trabajo (como `for`) hay una barrera implícita, salvo que se indique `nowait`¹; esto garantiza que todos los hilos terminen un `for` antes de continuar.

Las construcciones usadas en este proyecto son:

- `parallel for`: reparte las iteraciones de un ciclo entre los hilos del equipo.
- `collapse(n)`: fusiona n ciclos anidados en un único espacio de iteraciones.

- `schedule(static | dynamic | guided | runtime)`: define cómo se reparten las iteraciones; con `runtime` la decisión se toma al ejecutar mediante `omp_set_schedule`.
- `reduction(+ : var)`: cada hilo acumula en una copia privada y al final las copias se combinan.
- `simd`: pide al compilador vectorizar el ciclo.

Chapman, Jost y van der Pas destacan que OpenMP permite paralelizar de forma incremental: se parte de un programa secuencial correcto y se agregan directivas una por una², lo que es exactamente la estrategia iterativa que pide el proyecto.

2.2 Metodología PCAM

Foster propone diseñar algoritmos paralelos en cuatro etapas: **P**articionamiento (dividir el cómputo y los datos en tareas pequeñas), **C**omunicación (identificar qué datos necesita cada tarea de otras), **A**glomeración (agrupar tareas para reducir el costo de comunicación y de administración) y **M**apeo (asignar los grupos a procesadores)³. En la sección 3.3 se aplica al screensaver.

2.3 Speedup, eficiencia y ley de Amdahl

Si T_1 es el tiempo de la versión secuencial y T_p el de la paralela con p hilos:

- **Speedup**: $S(p) = T_1 / T_p$
- **Eficiencia**: $E(p) = S(p) / p$

La ley de Amdahl acota el speedup cuando una fracción f del trabajo es serial: $S(p) \leq 1 / (f + (1 - f)/p)$ ⁴. Invirtiéndola se obtiene la **fracción serial experimental** de Karp y Flatt, $e = (1/S - 1/p) / (1 - 1/p)$, que permite estimar f a partir del speedup medido: si e crece con p , la pérdida de eficiencia no viene del código serial sino del overhead de paralelización o de límites del hardware⁵.

2.4 SDL 2

Simple DirectMedia Layer es una biblioteca multiplataforma en C que da acceso de bajo nivel a video, audio y entrada⁶. Se usa solo para abrir la ventana, recibir eventos de teclado y subir a la tarjeta gráfica el arreglo de píxeles ya calculado mediante una textura de *streaming* (`SDL_TEXTUREACCESS_STREAMING`). La documentación de SDL advierte que las funciones de renderizado deben llamarse desde el hilo principal⁷, por lo que ninguna

llamada a SDL ocurre dentro de una región paralela. El texto de FPS se dibuja con la extensión SDL_ttf.

3. Cuerpo

3.1 Descripción del screensaver

El programa recibe como parámetro principal **N**, la **cantidad de fuentes de onda**. Cada fuente tiene un tono de color, frecuencia, amplitud, fase y una órbita (centro, radio, velocidad angular y ángulo inicial) generados de forma pseudoaleatoria con `std::mt19937` a partir de una semilla configurable. En cada frame:

1. Las fuentes avanzan en sus órbitas.
2. Para cada píxel se calculan coordenadas polares, se dobla el ángulo para producir la simetría de caleidoscopio y se transforma el radio en profundidad de túnel.
3. Se suman las **N** ondas en ese punto (interferencia) y el campo resultante determina anillos, radios, brillo y tono.
4. El color HSV se convierte a ARGB y se escribe en el búfer, que luego se sube a la ventana con una sola llamada a `SDL_UpdateTexture`.

El resultado (ver capturas del apéndice) es un túnel poligonal simétrico con anillos que avanzan hacia un centro brillante, en bandas de color que cambian a medida que las ondas se refuerzan o se cancelan. En la esquina superior izquierda se muestran los FPS, **N**, la resolución, la versión, los hilos y el tiempo de cálculo del frame.

3.2 Física y trigonometría del modelo

Movimiento orbital. La fuente *i* se mueve en una órbita elíptica deformada:

$$\text{posU} = \text{centroU} + \text{radioOrb} \cdot \cos(\text{velAng} \cdot t + \text{angIni})$$

$$\text{posV} = \text{centroV} + 0.35 \cdot \sin(1.3 \cdot \text{velAng} \cdot t + \text{angIni})$$

Coordenadas polares. Cada píxel (*x*, *y*) se normaliza respecto al centro de la pantalla y se convierte a polares: $r = \sqrt{(nx^2 + ny^2)}$, $\theta = \text{atan2}(ny, nx)$.

Simetría de caleidoscopio. Con $\text{sector} = 2\pi / \text{simetría}$, el ángulo se reduce a un sector, $m = \theta \bmod \text{sector}$, y se refleja respecto a su mitad: $v = |m - \text{sector}/2| / (\text{sector}/2)$. Así, todos los sectores muestran la misma imagen reflejada.

Proyección de túnel. La transformación $u = 0.55 / r + 0.35 t$ convierte la distancia al centro en profundidad: los píxeles cercanos al centro corresponden a zonas "lejanas" del túnel, y el término $0.35 t$ hace que el túnel avance con el tiempo.

Superposición de ondas. En el punto (u, v) , la distancia a la fuente i es $d_i = \sqrt{((u - \text{pos}U_i)^2 + (v - \text{pos}V_i)^2)}$ y su onda es

$$\text{onda}_i = \text{amplitud}_i \cdot \sin(2 \cdot \text{frecuencia}_i \cdot d_i - 1.4 t + \text{fase}_i)$$

El campo es el promedio de las N ondas. Donde las ondas llegan en fase se refuerzan (campo grande, anillos brillantes) y donde llegan en oposición se cancelan (zonas oscuras). Esta es la **interacción real entre los N elementos**: cada píxel depende de todas las fuentes a la vez. El tono resultante es el promedio de los tonos de las fuentes ponderado por $|\text{onda}_i|$, de modo que la fuente que más contribuye en un punto domina su color.

Luminosidad. Los anillos $(0.5 + 0.5 \sin(11u + 6 \cdot \text{campo}))$ y radios $(0.5 + 0.5 \sin(3\pi v + 4 \cdot \text{campo} + 0.6u))$ forman la estructura, y el brillo $0.22 + 1.05 / (1 + 2.2 r^2)$ hace que el punto de fuga sea más luminoso.

3.3 Diseño con PCAM

Particionamiento. Se usó descomposición de dominio sobre el búfer de píxeles: la tarea mínima es calcular el color de un píxel. Hay ancho $\times$ alto tareas (480×000 a 800×600), cada una con costo $O(N)$. Una segunda partición, mucho más pequeña, es la actualización de las N órbitas.

Comunicación. Las tareas de píxel no se comunican entre sí: cada una lee las N fuentes (datos compartidos de solo lectura) y escribe un único elemento del búfer. La única dependencia es que las fuentes deben estar actualizadas antes de pintar, y que el frame debe estar completo antes de subirlo a la ventana. Ambas se resuelven con barreras. La métrica global de brillo requiere combinar un valor por tarea, lo que se resuelve con una reducción.

Aglomeración. Un píxel es demasiado poco trabajo para repartirlo por separado. En v1 se aglomeran bloques contiguos de píxeles (`collapse(2)` con `static`); desde v2, **filas completas**, que tienen costo casi uniforme y ocupan memoria contigua, con lo que el falso compartimiento solo puede darse en las fronteras entre bloques de hilos.

Mapeo. Con carga uniforme, el mapeo estático (bloques de filas consecutivas por hilo) es el de menor overhead; se compararon además `dynamic` y `guided` con distintos tamaños de

bloque (barrido D).

3.4 Versión secuencial (v0)

screensaver_seq usa el mismo núcleo matemático (kaleidoscope.hpp/.cpp) que la paralela; las funciones del bucle caliente son inline y se compilan igual en ambos programas, así que la única diferencia medida es OpenMP. El frame secuencial (renderSecuencial) actualiza las N órbitas y recorre los píxeles con dos ciclos anidados. El motor común (motor.cpp) abre la ventana con renderer acelerado (con respaldo por software), crea la textura ARGB8888 y dibuja el texto de FPS con SDL_ttf.

3.5 Versiones paralelas

Las tres variantes son acumulativas y se eligen con --version:

v1 — Paralelización directa.

```
#pragma omp parallel for collapse(2) schedule(static)
for (int y = 0; y < p.alto; ++y)
    for (int x = 0; x < p.ancho; ++x) {
        double invR, v, r;                                // privadas por construcción
        polaresDePixel(x, y, p, invR, v, r);
        pixeles[y * p.ancho + x] = colorPixel(invR, v, r, fuentes, t);
    }
```

v2 — Scheduling y región paralela. Reparte filas completas con schedule(runtime) (se elige con --schedule y --chunk), agrega la métrica global de brillo con reduction(+:suma) y paraleliza la actualización de fuentes cuando $N \geq 256$ (con menos fuentes, crear el equipo cuesta más que lo que se ahorra).

v3 — Menos trabajo por píxel. r , v y $0.55/r$ solo dependen de (x, y) , así que se calculan una sola vez en una tabla (llenada con parallel for) y cada frame solo los lee: se eliminan sqrt, atan2 y fmod del bucle caliente. Además, cada frame abre **una sola** región paralela que contiene dos omp for (fuentes y píxeles), en lugar de dos regiones:

```
#pragma omp parallel reduction(+ : suma)
{
    #pragma omp for schedule(static)
    for (int i = 0; i < n; ++i) actualizarFuente(fuentes[i], t);
    // barrera implícita: todas las fuentes listas antes de pintar
    #pragma omp for schedule(runtime)
    for (int y = 0; y < p.alto; ++y) { /* lee la tabla, pinta y acumula suma */ }
}
```

v3-fast. La misma v3 compilada con `-O3 -march=native -ffast-math`. Con esas opciones se activa un `#pragma omp simd reduction(...)` en el ciclo sobre las N fuentes, y GCC vectoriza `sin` y `sqrt` con instrucciones AVX2. Se documenta aparte porque `-ffast-math` permite reordenar operaciones de punto flotante y, en general, cambia los resultados numéricos.

Para mantener la comparación honesta, ciertas constantes por fuente ($k = 2 \cdot \text{frecuencia}$ y $\text{desfase} = \text{fase} - 1.4t$) se precaculan una vez por frame **en todas las versiones**, incluida la secuencial; multiplicar por 2 es exacto en punto flotante, por lo que la salida no cambia.

3.6 Protección de la memoria compartida y sincronía

Dato	Clasificación	Justificación
Variables del píxel (<code>invR</code> , <code>v</code> , <code>r</code> , <code>campo</code> , ...)	privadas	Declaradas dentro del cuerpo del ciclo o de funciones <code>inline</code>
fuentes	compartida, solo lectura durante el pintado	Se escribe solo en la fase de actualización, separada por una barrera
píxeles	compartida, escritura en índices disjuntos	La iteración (<code>x</code> , <code>y</code>) solo escribe <code>píxeles[y·ancho + x]</code>
tabla (v3)	compartida, solo lectura	Se llena una vez antes del primer frame
suma	<code>reduction(+)</code>	Cada hilo acumula en una copia privada; OpenMP las combina al final

Por lo anterior no existen condiciones de carrera y **no se necesitan** `critical`, `atomic` ni **locks**. Los mecanismos de sincronía que sí se usan son:

1. **Barrera implícita al final de cada** `omp for`: en v3 separa la actualización de fuentes del pintado; en todas las versiones garantiza que el frame esté completo antes de `SDL_UpdateTexture`.

2. **Reducción** (`reduction(+:suma)`) para la métrica global de brillo. Como la suma es entera, el resultado paralelo es exactamente igual al secuencial y sirve como verificación.
3. **Fin de la región paralela (join)**: tras él solo continúa el hilo principal, que es el único que llama a SDL.

3.7 Programación defensiva y manejo de recursos

Todos los parámetros entran por línea de comandos; no hay valores fijos en el código. Una única función, `leerEntero`, valida cada entero con `strtol`, revisa que el puntero final llegue al terminador (rechaza `12abc`, `3.5` o cadenas vacías), detecta desbordamiento con `errno` y comprueba el rango. Se rechazan `--n 0`, valores negativos, resoluciones menores a 640×480 , schedules desconocidos, argumentos desconocidos, `--bench` con menos de 10 frames y `--csv` sin `--bench`; en cada caso se imprime un mensaje específico y el uso, y el programa termina con código 1 sin fallar.

Los recursos SDL se encapsulan en la clase `Ventana` (RAII): cada `SDL_Create*` se verifica y reporta con `SDL_GetError()`, y el destructor libera en orden inverso a la creación incluso si la creación se interrumpió. La memoria de píxeles, fuentes y tablas usa `std::vector`, y la tabla de `v3` vive en un `shared_ptr` capturado por la función del frame, de modo que no hay `new/delete` manuales.

3.8 Verificación de correctitud

El modo `--verify` fija `t = 3.7` y la semilla, calcula un frame con la versión secuencial y con `v1`, `v2` y `v3`, y compara los búferes. Resultados (detalle en el Anexo 3):

Configuración	Schedule	Hash FNV-1a v0	v1, v2, v3	Píxeles distintos	Dif. máx. por canal
N=64, 800×600 , simetría 8	static, dynamic,4, guided	e811ea536f4172b1	mismo hash	0	0
N=16, 1280×720 , simetría 12, semilla 7	static	aba0778f851b1612	mismo hash	0	0
N=32, 800×600	guided	eae9492a6c34c7ae	mismo hash	0	0

La suma entera `R+G+B` calculada con `reduction` en `v2` y `v3` coincide exactamente con la secuencial (por ejemplo, 93 422 142 para `N = 64`).

Las tres versiones paralelas son **idénticas bit a bit** a la secuencial en todas las configuraciones probadas, lo que confirma que no hay variables compartidas que debieran ser privadas.

3.9 Metodología de medición

- **Equipo:** AMD Ryzen 5 7520U (4 núcleos físicos, 8 hilos lógicos con SMT, 2.8–4.3 GHz), Windows 11 con WSL 2 (Ubuntu), g++ 15.2 con -O2, SDL 2.32. La laptop estuvo conectada a la corriente durante las pruebas.
- **Protocolo:** --bench corre sin ventana y mide solo el cálculo del frame con `std::chrono::steady_clock`, sin incluir la presentación. En cada configuración se descartan 5 frames de calentamiento y se miden **10 frames**; se reportan media, mediana, desviación estándar, mínimo, máximo y FPS equivalente ($1000 / \text{media}$).
- **Speedup y eficiencia** se calculan con el **promedio** de los tiempos, contra la versión secuencial v0 con el mismo N y la misma resolución.
- **Barridos:** A) hilos 1, 2, 4, 6 y 8 con $N = 64$ a 800×600 ; B) $N = 4, 16, 64, 256$ y 1024 a 800×600 con 8 hilos; C) resoluciones $640 \times 480, 800 \times 600, 1280 \times 720$ y 1920×1080 con $N = 64$; D) schedules; E) búsqueda del N máximo a 60 FPS en 640×480 (duplicando N y luego por bisección).

3.10 Resultados

El detalle completo ($66 \text{ configuraciones} \times 10 \text{ mediciones}$, con media, mediana, desviación, mínimo y máximo) está en el Anexo 3. Aquí se resumen los resultados principales.

Barrido A — speedup y eficiencia contra el número de hilos ($N = 64, 800 \times 600$, promedio de 10 mediciones; $T_{v0} = 1444 \text{ ms}$):

Hilos	v1 speedup	v1 eficiencia	v2 speedup	v2 eficiencia	v3 speedup	v3 eficiencia
1	0.93	0.93	0.91	0.91	1.04	1.04
2	1.94	0.97	2.31	1.15	2.10	1.05
4	4.61	1.15	3.90	0.97	3.99	1.00
6	5.00	0.83	5.13	0.85	5.46	0.91
8	4.50	0.56	6.06	0.76	5.81	0.73

Repetición intercalada (3 rondas de 10 mediciones cada una, $N = 64, 800 \times 600, 8 \text{ hilos}$), para separar diferencias reales del ruido de la máquina:

Versión	Schedule	Promedio (ms)	Speedup	Eficiencia
v0	—	1555.6	1.00	1.00
v1	static, collapse(2)	219.2	7.10	0.89
v3	static	199.3	7.81	0.98
v3	dynamic, 4	185.0	8.41	1.05
v3	guided	187.5	8.30	1.04

Barrido B — speedup contra N (800×600 , 8 hilos):

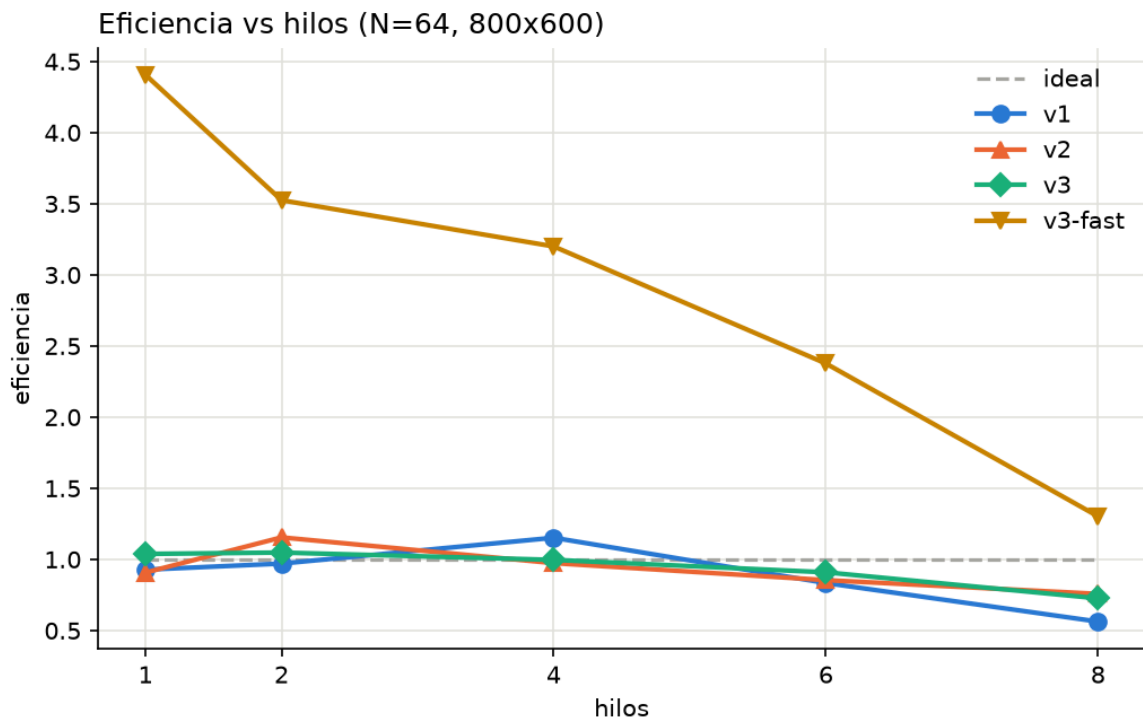
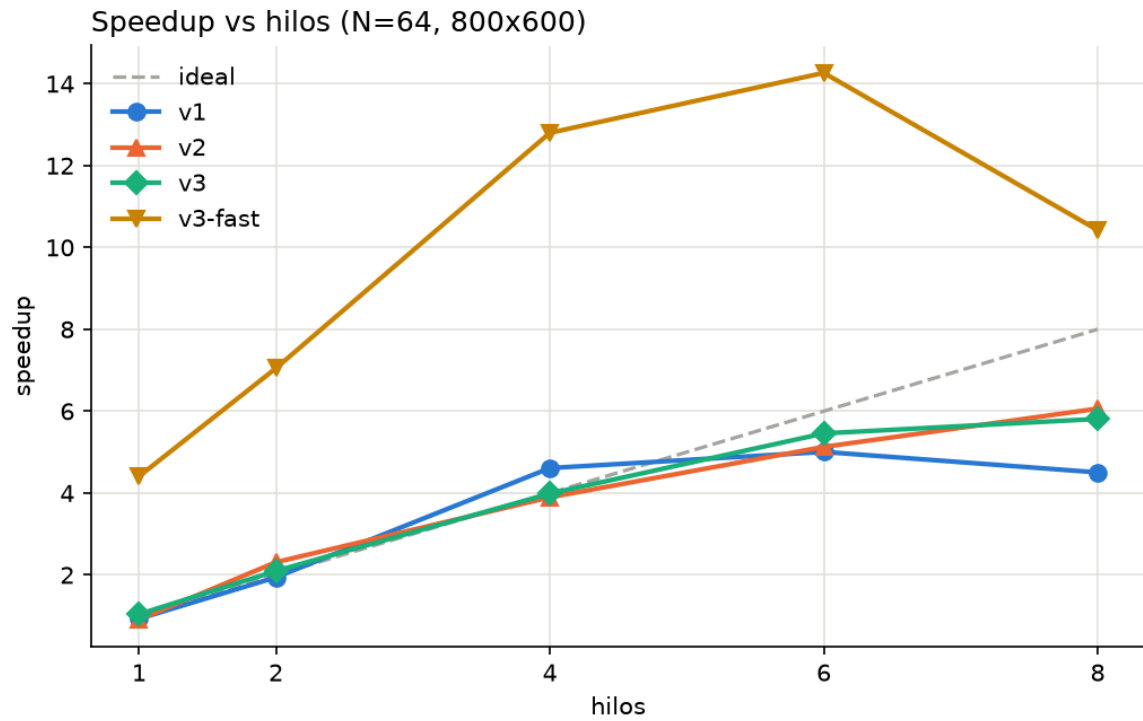
N	T_{v0} (ms)	v1	v2	v3	v3-fast
4	242	4.55	5.36	6.74	9.31
16	536	5.26	5.50	8.23	17.69
64	1444	4.50	6.06	5.81	10.42
256	6698	6.23	6.73	6.50	16.48
1024	28 979	10.13	8.59	8.18	21.41

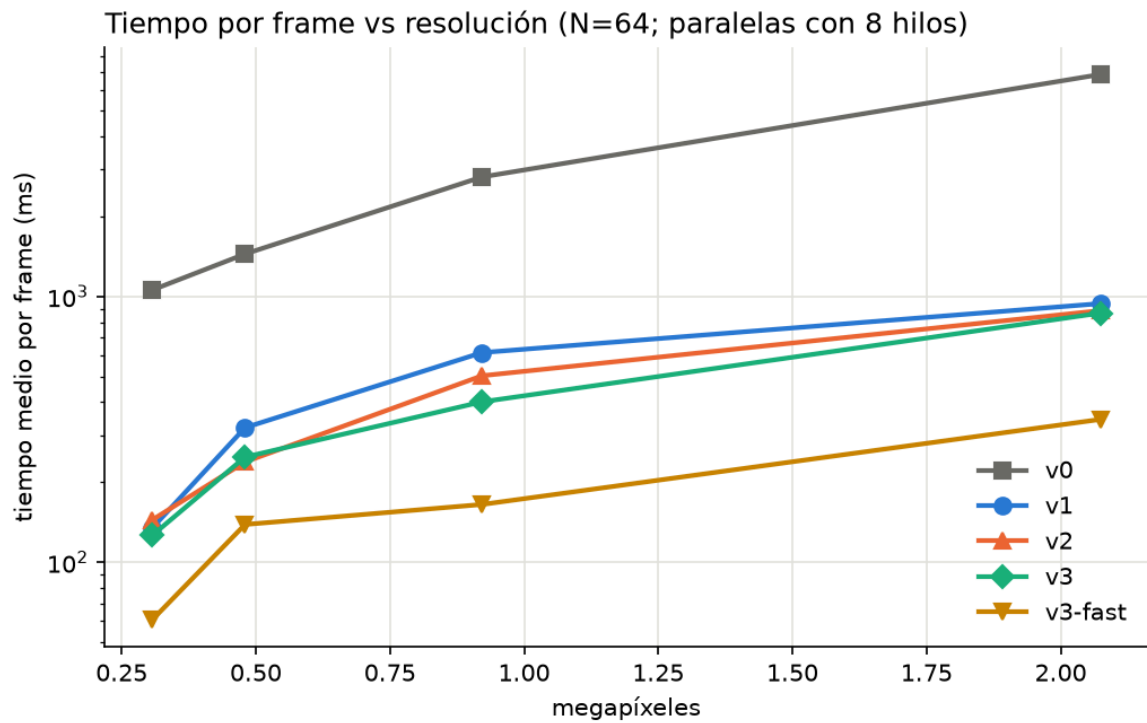
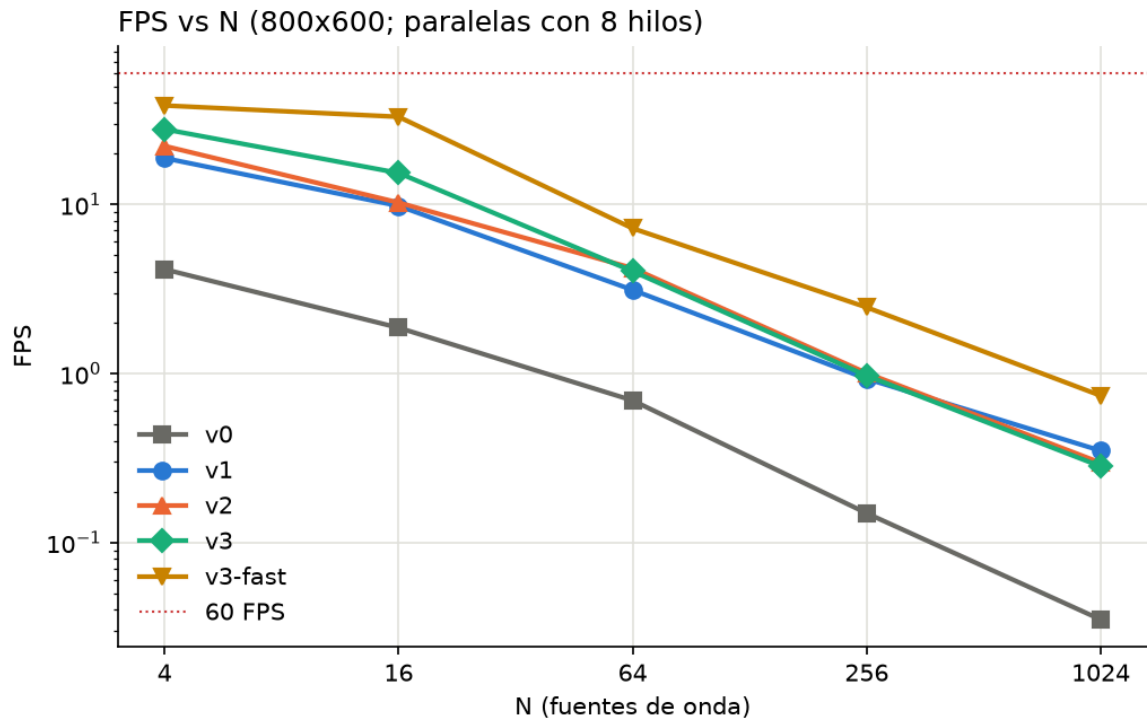
Barrido C — resolución ($N = 64$, 8 hilos): el tiempo crece en proporción al número de píxeles (v0: 1057 ms a 640×480 y 6842 ms a 1920×1080). A 1920×1080 los speedups fueron 7.28 (v1), 7.74 (v2), 7.91 (v3) y 19.87 (v3-fast).

Barrido D — schedules (v2 y v3, $N = 64$, 800×600 , 8 hilos): guided fue el más rápido en ambas versiones (198 ms en v2 y 178 ms en v3, contra 238 y 249 ms con static). dynamic con chunk 1, 4 y 16 quedó en medio.

N máximo que sostiene 60 FPS (640×480 , el canvas mínimo; búsqueda por duplicación y bisección):

Versión	N máximo a ≥ 60 FPS	FPS con ese N	FPS con $N = 1$
v0 secuencial	no alcanza	—	7.9
v1 (8 hilos)	no alcanza	—	58.7
v2 (8 hilos)	no alcanza	—	56.7
v3 (8 hilos)	3	69.5	91.3
v3-fast (8 hilos)	16	60.7	117.3





3.11 Análisis

Escalamiento casi lineal hasta 4 hilos. Con 2 y 4 hilos las tres versiones obtienen speedups de 1.9–2.3 y 3.9–4.6, con eficiencias cercanas a 1. Es lo esperado para un problema con paralelismo de datos: los píxeles son independientes, no hay secciones críticas y la única parte serial dentro del cálculo es la actualización de las N órbitas, que cuesta N evaluaciones de $\sin/\cos$ frente a $N \times 480\,000$ del pintado. Algunas eficiencias superan ligeramente 1 (p. ej. 1.15); se deben a la variabilidad de la línea base secuencial, cuya desviación estándar llegó a 21 % de la media (frecuencia del procesador que cambia con la temperatura y la carga de Windows).

Ley de Amdahl y fracción serial. La fracción serial de Karp-Flatt medida con 4 hilos es casi nula (0.009 en v2 y 0.001 en v3), y sube a 0.05 con 8 hilos. Si la parte serial fuera la causa, e sería constante; como crece con p , la pérdida de eficiencia viene del hardware y no del código serial. Con $f \approx 0.01$, Amdahl predice un máximo de $1/(0.01 + 0.99/8) = 7.5$ con 8 hilos, cercano a lo que se obtuvo en la repetición (7.1–8.4). En el modo interactivo sí aparece una parte serial importante: la subida de la textura y la presentación (SDL, hilo principal). Con v3, $N = 14$ y 8 hilos, el cálculo tomó 59.6 ms y la pantalla mostró 13.2 FPS (75.8 ms por frame), así que ~ 16 ms por frame son seriales; eso equivale a $f \approx 16 / (16 + 8 \cdot 59.6) \approx 3\%$ y limita el speedup del programa completo a $\sim 1/f \approx 30$ aunque hubiera infinitos hilos.

Caída de eficiencia de 4 a 8 hilos: SMT. El procesador tiene 4 núcleos físicos; los hilos 5 a 8 comparten núcleo con otro hilo (hyperthreading/SMT) y por lo tanto sus unidades de ejecución. Aun así, el speedup siguió subiendo hasta ~ 6 –8, lo que indica que un solo hilo no satura su núcleo: el ciclo sobre las N fuentes es una cadena de operaciones dependientes ($\text{sqrt} \rightarrow \sin \rightarrow \text{acumulación}$) limitada por latencia, y el segundo hilo SMT aprovecha las unidades que quedan ociosas. Por eso la eficiencia "por hilo lógico" baja a 0.7–0.9, pero la eficiencia por núcleo físico sigue siendo alta. El ancho de banda de memoria no es un límite: cada frame escribe menos de 2 MB y todo lo demás cabe en caché.

El speedup mejora al aumentar N . En el barrido B, v2 pasa de 5.4 ($N = 4$) a 8.6 ($N = 1024$). Con N grande, cada píxel hace mucho más trabajo por la misma cantidad de overhead fijo (crear el equipo de hilos, repartir filas, barrera final y desbalance del último bloque), así que la razón cómputo/overhead crece. Además, el ciclo de ondas es justamente la parte limitada por latencia que más se beneficia de SMT. El 10.1 de v1 con $N = 1024$ supera el número de hilos lógicos y no es físicamente posible como speedup "puro":

muestra el ruido de la línea base (un frame secuencial de 29 s acumula variaciones de frecuencia); en la repetición controlada ningún speedup pasó de 8.4.

Mejora iterativa $v1 \rightarrow v2 \rightarrow v3$.

- $v1$ con 1 hilo es 7 % más lenta que la secuencial: `collapse(2)` obliga a recuperar (x, y) del índice fusionado con divisiones en cada iteración. Con 8 hilos quedó por debajo de $v2$ y $v3$ (219 ms contra 185–199 ms en la repetición).
- $v2$ reparte filas completas, que son bloques contiguos de memoria; su ventaja sobre $v1$ es modesta pero constante, y su `reduction` confirma la correctitud.
- $v3$ elimina `sqrt`, `atan2` y `fmod` por píxel. Esa ganancia es fija por píxel, mientras que el trabajo de las ondas crece con N : por eso $v3$ es 20 % más rápida que $v2$ con $N = 4$ y 33 % con $N = 16$, pero con $N \geq 64$ la diferencia queda dentro del ruido. Es la versión que permite llegar a 60 FPS.
- $v3$ -fast (misma $v3$ con `-O3 -march=native -ffast-math`) es 2.2–2.7 veces más rápida que $v3$ con $N \geq 64$ porque GCC vectoriza `sin` y `sqrt` con AVX2 (4 fuentes por instrucción). Su "speedup" de 10–21 mezcla dos efectos distintos, paralelización y vectorización, por eso su eficiencia supera 1.

Schedules. Aunque todas las filas hacen la misma cantidad de operaciones, `guided` y `dynamic`,⁴ resultaron 6–7 % más rápidos que `static` en la repetición, y `guided` también fue el mejor en el barrido D. La explicación es que el costo real de las filas no es idéntico (`pow` y `sin` tardan distinto según el valor de entrada) y, sobre todo, que la laptop ejecuta otros procesos: con `static`, un hilo interrumpido por el sistema operativo retrasa todo el frame, porque la barrera final espera al más lento; con reparto dinámico los demás hilos absorben su trabajo. `guided` empieza con bloques grandes (poco overhead) y termina con bloques pequeños (buen balance), así que se dejó como valor por defecto (`--schedule guided`). `dynamic` con `chunk 1` no fue desastroso, como sí ocurre con ciclos baratos, porque cada fila cuesta cientos de microsegundos y pedir la siguiente es despreciable en comparación.

N máximo a 60 FPS. Calcular el color en el CPU es caro: incluso con $N = 1$, cada píxel cuesta unos 490 ns en la versión secuencial (`atan2`, tres `pow`, varios `sin` y la conversión HSV), así que $v0$ no pasa de 7.9 FPS a 640×480 y nunca llega a 60 FPS. $v1$ y $v2$ se quedan justo debajo (57–59 FPS con $N = 1$). $v3$, al quitar la trigonometría polar del bucle, sostiene 60 FPS hasta $N = 3$, y $v3$ -fast hasta $N = 16$. Para valores mayores de N el programa sigue siendo fluido en términos relativos: con $N = 64$ a 800×600 , $v3$ da ~5.3 FPS contra 0.69 de la secuencial (7.8 veces más).

4. Conclusiones

1. La versión secuencial y las tres versiones paralelas despliegan el screensaver correctamente. `--verify` mostró que `v1`, `v2` y `v3` producen búferes **idénticos bit a bit** a la secuencial con todos los schedules probados, lo que confirma que la clasificación de variables (privadas, compartidas de solo lectura, escritura en índices disjuntos y reduction) es correcta y que no se necesitan `critical` ni `atomic`.
2. La paralelización con OpenMP escaló casi linealmente hasta los 4 núcleos físicos (speedup 3.9–4.6, eficiencia ≈ 1) y llegó a speedups de 7.1–8.4 con 8 hilos lógicos (eficiencia 0.89–1.05) en la repetición controlada, gracias a que SMT aprovecha las unidades ociosas de un cálculo limitado por latencia.
3. La fracción serial del cálculo es menor al 1 %; la pérdida de eficiencia con más hilos se explica por el hardware (SMT) y por el ruido del sistema operativo, no por código serial. En el modo interactivo, la presentación con SDL (serial por diseño) representa ~ 3 % del frame y es el siguiente límite según la ley de Amdahl.
4. El speedup aumenta con N porque crece la razón entre cómputo y overhead fijo por frame.
5. Las mejoras iterativas funcionaron en el régimen esperado: repartir filas (`v2`) y usar `guided` mejoró el balance frente a `collapse(2)`; precalcular la tabla polar (`v3`) redujo el tiempo 20–33 % con N pequeño. Solo `v3` y `v3-fast` alcanzan 60 FPS a 640×480 , con N máximo de 3 y 16 respectivamente.

5. Recomendaciones

1. Medir en un equipo de escritorio con frecuencia fija (o con el modo de energía en "Máximo rendimiento") y con menos procesos en segundo plano: la desviación de la línea base secuencial (hasta 21 %) produjo algunos speedups aparentes mayores que el número de hilos.
2. Reportar también la mediana o el mínimo de las mediciones, que son menos sensibles a interrupciones del sistema operativo que el promedio.
3. Para aumentar N a 60 FPS, reducir el trabajo por píxel: guardar las fuentes como estructura de arreglos (SoA) para vectorizar sin `-ffast-math`, reemplazar las tres llamadas a `pow` por tablas o aproximaciones, o calcular a menor resolución y escalar la textura.
4. Usar `-ffast-math` solo después de comparar con `--verify`: en estas pruebas no cambió ningún píxel, pero no hay garantía para otras semillas o compiladores.

5. Explorar una región parallel persistente entre frames (el hilo maestro hace las llamadas a SDL entre dos barreras) para eliminar la creación del equipo de hilos en cada frame, y superponer el cálculo del frame siguiente con la presentación del actual (doble búfer).

6. Referencias

Notas al pie

1. OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*, sección 11.5 "Worksharing-Loop Constructs". <https://www.openmp.org/specifications/>
2. Chapman, B., Jost, G. y van der Pas, R. (2008). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, cap. 1.
3. Foster, I. (1995). *Designing and Building Parallel Programs*. Addison-Wesley, cap. 2 "Designing Parallel Algorithms". <https://www.mcs.anl.gov/~itf/dbpp/>
4. Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30, 483–485.
5. Pacheco, P. y Malensek, M. (2022). *An Introduction to Parallel Programming* (2.^a ed.). Morgan Kaufmann, cap. 2 y 5.
6. SDL Project. (2026). *Simple DirectMedia Layer*. <https://www.libsdl.org/>
7. SDL Wiki. (2026). *CategoryRender*: las funciones de renderizado solo deben llamarse desde el hilo principal. <https://wiki.libsdl.org/SDL2/CategoryRender>

Bibliografía

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30, 483–485.
- Chapman, B., Jost, G. y van der Pas, R. (2008). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.
- Foster, I. (1995). *Designing and Building Parallel Programs*. Addison-Wesley. <https://www.mcs.anl.gov/~itf/dbpp/>
- Karp, A. H. y Flatt, H. P. (1990). Measuring parallel processor performance. *Communications of the ACM*, 33(5), 539–543.
- OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*. <https://www.openmp.org/specifications/>
- Pacheco, P. y Malensek, M. (2022). *An Introduction to Parallel Programming* (2.^a ed.). Morgan Kaufmann.
- SDL Project. (2026). *Simple DirectMedia Layer* y *SDL2 Wiki*. <https://www.libsdl.org/> — <https://wiki.libsdl.org/SDL2/>

7. Apéndices

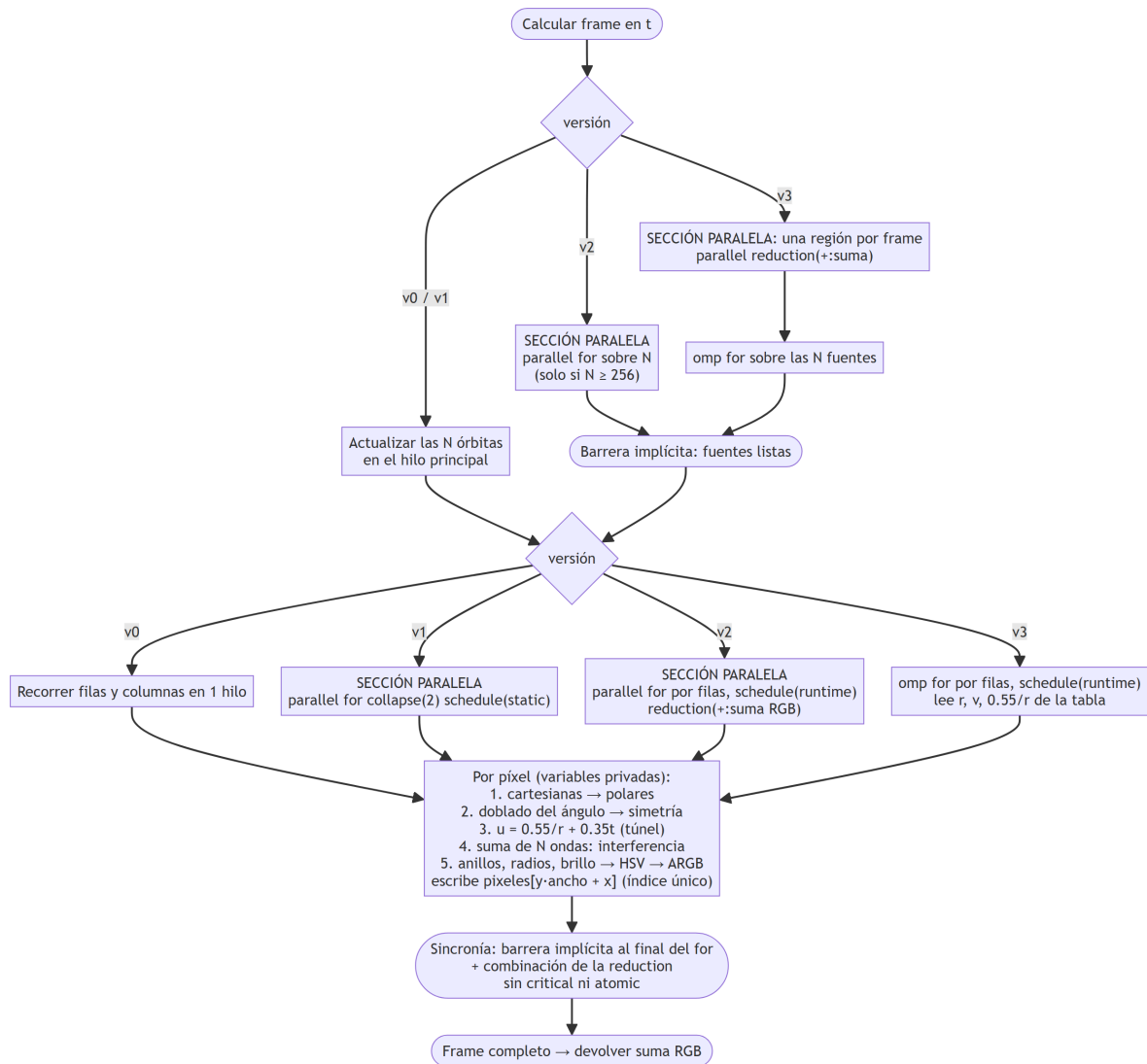
- **Anexo 1 — Diagrama de flujo:** `anexo1_diagrama_flujo.png` (programa completo) y `anexo1_diagrama_frame.png` (cálculo de un frame y sincronía).
- **Anexo 2 — Catálogo de funciones:** `anexo2_catalogo.md`.
- **Anexo 3 — Bitácora de pruebas:** `anexo3_bitacora.md`, con las 10 mediciones de cada prueba, speedups, eficiencias, gráficas y capturas.

Anexo 1 — Diagrama de flujo

El diagrama se divide en dos partes para que sea legible. La figura A1 muestra el programa completo: captura de argumentos, validación (programación defensiva), configuración de OpenMP, los tres modos de ejecución (verificación, benchmark e interactivo) y el despliegue de resultados. La figura A2 detalla el cálculo de un frame en cada versión, con las secciones paralelas y los mecanismos de sincronía (barreras implícitas y reducción).

Fuentes en Mermaid: `anexo1_diagrama_flujo.mmd` y `anexo1_diagrama_frame.mmd` (también exportadas a SVG).

Figura A1. Flujo general del programa (secuencial y paralelo).



Leyenda: los rectángulos son procesos, los rombos son decisiones, los paralelogramos son entradas y salidas, y los óvalos de la figura A2 son puntos de sincronización. "Sección paralela" marca las regiones donde trabaja el equipo de hilos de OpenMP; todas las llamadas a SDL ocurren fuera de ellas, en el hilo principal.

Anexo 2 — Catálogo de funciones

Proyecto 1 · Screensaver "Túnel Caleidoscópico de Interferencia" · Computación Paralela y Distribuida, UVG 2026

Integrantes: Carlos Daniel Estrada Vega (20853), André Emilio Pivaral López (23574), Daniela Ramírez de León (23053)

El código está dividido en cinco archivos:

Archivo	Contenido
src/kaleidoscope.hpp	Estructuras de datos y funciones del bucle caliente (<code>inline</code>), compartidas por todas las versiones
src/kaleidoscope.cpp	Validación de argumentos, creación de fuentes, frame secuencial v0, hash y estadísticas
src/motor.hpp/.cpp	Ventana SDL (RAII), ciclo interactivo con FPS y modo benchmark
src/screensaver_seq.cpp	Programa secuencial (v0)
src/screensaver_par.cpp	Programa paralelo: v1, v2, v3 y modo <code>--verify</code>

1. Estructuras y clases

`struct Fuente` — `kaleidoscope.hpp`

Propósito: representa una de las N fuentes de onda. Cada fuente orbita en el plano (u, v) y emite una onda senoidal; la superposición de todas define el color de cada píxel.

Campo	Tipo	Uso
hue	double	Tono base del color, en [0, 1)
frecuencia	double	Frecuencia espacial de la onda
amplitud	double	Amplitud de la onda
fase	double	Desfase inicial
radioOrb, velAng, angIni	double	Radio, velocidad angular y ángulo inicial de la órbita
centroU, centroV	double	Centro de la órbita
posU, posV	double	Posición actual (se recalcula cada frame)
k	double	$2 \cdot \text{frecuencia}$ (constante, se calcula una vez)

Campo	Tipo	Uso
desfase	double	fase – 1.4 t (se recalcula cada frame)

struct Parametros — kaleidoscope.hpp

Propósito: agrupa todos los parámetros de ejecución leídos de la línea de comandos. No hay valores fijos en el código: cada campo tiene un valor por defecto que el usuario puede cambiar.

Campo	Tipo	Uso
n	int	Cantidad de fuentes de onda (obligatorio, 1–4096)
ancho, alto	int	Resolución del canvas (mín. 640×480)
simetria	int	Orden de simetría del caleidoscopio (3–24)
semilla	int	Semilla del generador pseudoaleatorio
frames	int	Frames a ejecutar (0 = infinito); en --bench, frames medidos
version, hilos, chunk	int	Variante paralela, hilos OpenMP y tamaño de bloque
schedule	std::string	static, dynamic o guided
csv, captura	std::string	Archivo de resultados del benchmark y BMP de captura
fullscreen, bench, verify	bool	Modos de ejecución

struct TablaPolar — kaleidoscope.hpp

Propósito: guarda por píxel los valores que solo dependen de (x, y) y no del tiempo, para que v3 no los recalcula cada frame.

Campo	Tipo	Uso
invR	std::vector<double>	0.55 / r (profundidad del túnel)
v	std::vector<double>	Ángulo doblado, en [0, 1]
r	std::vector<double>	Radio normalizado

struct Estadisticas — kaleidoscope.hpp

Propósito: resumen estadístico de una serie de tiempos. Campos double: media, mediana, desv (desviación estándar muestral), minimo, maximo, todos en milisegundos.

```
struct InfoVersion — motor.hpp
```

Propósito: describe la versión que se ejecuta para el texto en pantalla y el CSV. Campos: nombre (std::string, "v0"... "v3"), hilos (int), schedule (std::string).

```
using FuncionFrame — motor.hpp
```

Propósito: tipo std::function<long long(double t, std::vector<uint32_t>& pixeles)>. Es la interfaz común entre el motor y cada versión: recibe el instante t, llena el búfer y devuelve la suma R+G+B del frame (o -1 si la versión no la calcula).

```
class Ventana — motor.cpp
```

Propósito: encapsula los recursos SDL (ventana, renderer, textura, fuente TTF y textura de texto) con RAII. El constructor no crea nada; iniciar crea y verifica cada recurso; el destructor los libera en orden inverso a su creación, aunque la creación se haya interrumpido a la mitad. No se puede copiar (evita liberar dos veces).

Método	Entradas	Salidas	Descripción
iniciar	p: const Parametros& — resolución y pantalla completa	bool — true si todo se creó	Inicializa SDL, crea la ventana y un renderer acelerado (o software si falla), la textura de streaming ARGB8888 y carga una fuente TTF. Si la fuente no carga, el FPS se mostrará en el título. Reporta errores con SDL_GetError().
procesarEventos	—	bool — false si hay que salir	Vacía la cola de eventos; devuelve false ante SDL_QUIT, ESC o Q.
mostrar	pixeles: const std::vector<uint32_t>& — frame calculado; texto: const std::string& — línea de FPS; captura: const std::string& — ruta BMP o vacío	—	Sube el búfer con una sola SDL_UpdateTexture, lo copia al renderer, dibuja el texto, guarda la captura si se pidió y presenta.
dibujarTexto (privado)	texto: const std::string&	—	Regenera la textura del texto solo si cambió y la dibuja sobre un fondo semitransparente; sin fuente, usa SDL_SetWindowTitle.

Método	Entradas	Salidas	Descripción
guardarCaptura (privado)	ruta: const std::string&	archivo BMP	Lee el contenido del renderer con SDL_RenderReadPixels y lo guarda con SDL_SaveBMP.
fallo (privado)	funcion: const char*	bool (siempre false)	Imprime el nombre de la función SDL que falló y SDL_GetError().

2. Validación de argumentos (programación defensiva) —

kaleidoscope.cpp

leerEntero

	Nombre	Tipo	Uso
Entrada	nombre	const char*	Nombre del argumento, para el mensaje de error
Entrada	texto	const char*	Texto a convertir (puede ser nullptr si faltó el valor)
Entrada	minimo, maximo	long	Rango válido, inclusivo
Salida	destino	int&	Valor convertido (solo se escribe si es válido)
Salida	retorno	bool	true si el valor es válido

Descripción: única función de validación de enteros. Usa strtol y revisa el puntero final (rechaza "12abc", "", "3.5"), errno == ERANGE (desbordamiento) y el rango. Cada falla imprime un mensaje específico.

leerParametros

	Nombre	Tipo	Uso
Entrada	argc, argv	int, char**	Argumentos del programa
Entrada	esParalela	bool	Habilita --version, --hilos, --schedule, --chunk, --verify
Salida	p	Parametros&	Parámetros leídos
Salida	retorno	bool	false si hubo error (el programa termina con código 1)

Descripción: recorre los argumentos; los enteros pasan por leerEntero con su rango, --schedule se compara contra los tres valores válidos, y cualquier argumento desconocido

produce el mensaje de uso. Al final revisa que `--n` esté presente, que `--bench` tenga al menos 10 frames medidos (20 por defecto) y que `--csv` solo se use con `--bench`.

`imprimirUso`

Entradas: programa (`const char*`, nombre del ejecutable), `esParalela` (`bool`). **Salida:** texto en `stderr`. **Descripción:** imprime la sintaxis, los rangos y los valores por defecto de cada parámetro.

3. Modelo y cálculo del frame

`crearFuentes` — `kaleidoscope.cpp`

	Nombre	Tipo	Uso
Entrada	<code>n</code>	<code>int</code>	Cantidad de fuentes
Entrada	<code>semilla</code>	<code>unsigned</code>	Semilla del <code>std::mt19937</code>
Salida	<code>retorno</code>	<code>std::vector<Fuente></code>	Fuentes inicializadas

Descripción: genera los parámetros de cada fuente con una distribución uniforme $U(0,1)$ según la especificación (p. ej. $frecuencia = 1.5 + 3.0 \cdot U$). El orden de las llamadas es fijo, así que la misma semilla produce siempre las mismas fuentes: es lo que permite comparar la salida secuencial con la paralela.

`actualizarFuente` — `kaleidoscope.hpp` (**inline**)

Entradas: `f` (`Fuente&`), `t` (`double`, segundos). **Salida:** `f.posU`, `f.posV`, `f.desfase` actualizados.

Descripción: movimiento orbital con trigonometría: $posU = centroU + radioOrb \cdot \cos(velAng \cdot t + angIni)$, $posV = centroV + 0.35 \cdot \sin(1.3 \cdot velAng \cdot t + angIni)$; también precalcula $fase - 1.4t$ para no repetirlo por píxel.

`polaresDePixel` — `kaleidoscope.hpp` (**inline**)

	Nombre	Tipo	Uso
Entrada	<code>x, y</code>	<code>int</code>	Coordenadas del píxel
Entrada	<code>p</code>	<code>const Parametros&</code>	Resolución y simetría
Salida	<code>invR</code>	<code>double&</code>	$0.55 / r$

	Nombre	Tipo	Uso
Salida	v	double&	Ángulo doblado en [0, 1]
Salida	r	double&	Radio normalizado (mínimo 1e-3)

Descripción: normaliza el píxel respecto al centro, lo convierte a polares con `sqrt` y `atan2`, dobla el ángulo con `fmod` dentro de un sector de $2\pi/\text{simetria}$ (simetría de caleidoscopio) y aplica la proyección $1/r$ que da la profundidad del túnel.

`colorPixel` — `kaleidoscope.hpp` (**inline**)

	Nombre	Tipo	Uso
Entrada	invR, v, r	double	Coordenadas del píxel (de <code>polaresDePixel</code> o de la tabla)
Entrada	fuentes	const std::vector<Fuente>&	Fuentes de onda (solo lectura)
Entrada	t	double	Tiempo
Salida	retorno	uint32_t	Color ARGB8888

Descripción: núcleo del screensaver. Calcula $u = 0.55/r + 0.35t$, suma las N ondas $\text{amplitud} \cdot \sin(k \cdot d + \text{desfase})$ (d = distancia del píxel a la fuente: **interferencia**), pondera el tono de cada fuente por $|\text{onda}|$, construye anillos y radios, aplica un brillo que crece hacia el punto de fuga y convierte HSV a RGB. Todas sus variables son locales, así que es seguro llamarla desde varios hilos a la vez. En el build `-ffast-math` el bucle sobre N lleva `#pragma omp simd`.

`hsvARgb` — `kaleidoscope.hpp` (**inline**)

Entradas: h, s, val (double, en [0, 1]). **Salida:** `uint32_t` ARGB con alfa 255. **Descripción:** conversión estándar $HSV \rightarrow RGB$ por sextantes del círculo de color, redondeando cada canal a 0–255.

`sumaCanales` — `kaleidoscope.hpp` (**inline**)

Entrada: `argb` (`uint32_t`). **Salida:** `int`, $R + G + B$. **Descripción:** su total por frame es la métrica global de brillo; al ser entera, la suma paralela con `reduction` da exactamente el mismo valor que la secuencial.

renderSecuencial (v0) — kaleidoscope.cpp

	Nombre	Tipo	Uso
Entrada/salida	fuentes	<code>std::vector<Fuente>&</code>	Se actualizan sus posiciones
Salida	pixeles	<code>std::vector<uint32_t>&</code>	Frame calculado
Entrada	p, t	<code>const Parametros&, double</code>	Parámetros y tiempo
Salida	retorno	<code>long long</code>	Suma R+G+B del frame

Descripción: actualiza las N órbitas y recorre todos los píxeles en un solo hilo. Es la referencia de tiempo y de correctitud.

renderV1 — screensaver_par.cpp

Entradas/salidas: las mismas que `renderSecuencial`; devuelve `-1` (no calcula la métrica).

Descripción: actualiza las fuentes en el hilo principal y paraleliza el bucle de píxeles con `#pragma omp parallel for collapse(2) schedule(static)`. Las variables del píxel se declaran dentro del bucle (privadas), las fuentes son de solo lectura y cada iteración escribe un índice distinto, por lo que no se necesita `critical` ni `atomic`.

renderV2 — screensaver_par.cpp

Entradas/salidas: las mismas que `renderSecuencial`. **Descripción:** actualiza las fuentes con `parallel for` cuando $N \geq 256$ (`MIN_FUENTES_PARALELO`), reparte filas completas con `schedule(runtime)` (elegido con `--schedule/--chunk`) y acumula el brillo con `reduction(+:suma)`.

crearTablaPolar — screensaver_par.cpp

Entrada: p (`const Parametros&`). **Salida:** `TablaPolar`. **Descripción:** llena en paralelo (`parallel for schedule(static)`) las tablas `invR`, `v` y `r` llamando a `polaresDePixel`, una sola vez al iniciar.

renderV3 — screensaver_par.cpp

Entradas/salidas: las de `renderSecuencial` más `tabla` (`const TablaPolar&`). **Descripción:** abre una sola región `parallel reduction(+:suma)` por frame; dentro, un `omp for` actualiza las fuentes y, tras su barrera implícita, otro `omp for schedule(runtime)` pinta las filas leyendo la tabla, sin `sqrt`, `atan2` ni `fmod` por píxel.

configurarSchedule — screensaver_par.cpp

Entrada: p (const Parametros&). **Salida:** estado de OpenMP. **Descripción:** traduce --schedule y --chunk a omp_set_schedule, que usan v2 y v3 vía schedule(runtime). Chunk 0 = valor por defecto.

etiquetaSchedule — screensaver_par.cpp

Entrada: p. **Salida:** std::string como "static", "dynamic:4" o "static-collapse2". **Descripción:** texto del schedule para la consola y el CSV.

crearFrame — screensaver_par.cpp

	Nombre	Tipo	Uso
Entrada	version	int	1, 2 o 3
Entrada	fuentes, p	std::vector<Fuente>&, const Parametros&	Estado capturado por referencia
Salida	retorno	FuncionFrame	Función que calcula un frame con esa versión

Descripción: devuelve una lambda que llama a la versión elegida; para v3 crea la tabla polar y la guarda en un shared_ptr dentro de la lambda, de modo que se libera sola cuando la lambda deja de existir.

verificar — screensaver_par.cpp

Entrada: p (const Parametros&). **Salida:** int (0 si las tres versiones son idénticas a la secuencial, 2 si no) y un reporte en consola. **Descripción:** calcula un frame en $t = 3.7$ con v0 y con v1, v2, v3 (cada una con fuentes nuevas de la misma semilla) y reporta píxeles distintos, la diferencia máxima por canal y el hash FNV-1a de cada búfer.

4. Motor de ejecución — motor.cpp

ejecutarInteractivo

	Nombre	Tipo	Uso
Entrada	p	const Parametros&	Resolución, frames, captura

	Nombre	Tipo	Uso
Entrada	info	const InfoVersion&	Versión e hilos para el texto
Entrada	frame	const FuncionFrame&	Cálculo del frame
Salida	retorno	int	Código de salida (1 si SDL falla)

Descripción: crea la Ventana; en cada vuelta procesa eventos, calcula el frame con $t =$ segundos transcurridos, mide el tiempo de cálculo y cada 0.5 s actualiza el texto "FPS | N | resolución | versión | hilos | cálculo ms"; luego muestra el frame desde el hilo principal. Al salir imprime el FPS promedio.

`ejecutarBenchmark`

Entradas: las mismas que `ejecutarInteractivo`. **Salidas:** int (1 si no pudo abrir el CSV), línea de resultados en consola y una fila en el CSV. **Descripción:** sin ventana; ejecuta 5 frames de calentamiento y luego `p.frames` frames con $t = i/60$, cronometrando solo el cálculo con `std::chrono::steady_clock`. Calcula media, mediana, desviación, mínimo, máximo y FPS equivalente, imprime todas las muestras y agrega la fila al CSV (con encabezado si el archivo es nuevo).

`msDesde`

Entrada: inicio (`steady_clock::time_point`). **Salida:** double, milisegundos transcurridos.

5. Utilidades — `kaleidoscope.cpp`

`hashFNV1a`

Entrada: pixeles (`const std::vector<uint32_t>&`). **Salida:** `uint64_t`. **Descripción:** hash FNV-1a de 64 bits sobre los bytes del búfer; dos frames idénticos tienen el mismo hash.

`calcularEstadisticas`

Entrada: tiempos (`std::vector<double>`, copia que se ordena). **Salida:** Estadísticas. **Descripción:** media, mediana, desviación estándar muestral (divisor $n - 1$), mínimo y máximo.

6. Programas principales

main — screensaver_seq.cpp

Entradas: argc, argv. **Salida:** código de salida. **Descripción:** valida los parámetros, crea las fuentes, envuelve renderSecuencial en una FuncionFrame y ejecuta el modo interactivo o --bench.

main — screensaver_par.cpp

Entradas: argc, argv. **Salida:** código de salida. **Descripción:** valida los parámetros, fija los hilos y el schedule de OpenMP, atiende --verify, crea el frame de la versión elegida y ejecuta el modo interactivo o --bench. En el build -ffast-math la versión se etiqueta con el sufijo -fast.

7. Scripts

Script	Entradas	Salidas	Descripción
bench/run_bench.sh	Ejecutables compilados; variable MEDICIONES (10 por defecto)	bench/results/crudo.csv, bitacora.log, n_max_60fps.csv, busqueda_60fps.csv	Verificación de correctitud y barridos de hilos, N, resolución y schedule; búsqueda (duplicación + bisección) del N máximo a 60 FPS.
bench/plot_results.py	crudo.csv, n_max_60fps.csv	resultados.csv, tablas.md, gráficas PNG	Calcula $\text{speedup} = T_o / T_p$ (promedios) y eficiencia = $\text{speedup} / p$, la fracción serial de Karp-Flatt, y genera las tablas y gráficas.

Anexo 3 — Bitácora de pruebas

Entorno y protocolo

Procesador	AMD Ryzen 5 7520U (4 núcleos físicos, 8 hilos lógicos con SMT; 2.8 GHz base, hasta 4.3 GHz)
Sistema	Windows 11 Pro + WSL 2 (Ubuntu), laptop conectada a la corriente, plan de energía "Equilibrado"
Compilador	g++ 15.2.0; -O2 -std=c++17 -Wall -Wextra (+ -fopenmp en la paralela); v3-fast: -O3 -march=native -ffast-math
Bibliotecas	SDL 2.32.10, SDL_ttf 2.24.0
Fecha	23 de septiembre de 2026

- Comando: `bash bench/run_bench.sh` y luego `python3 bench/plot_results.py`. La salida completa de la consola está en `bench/results/bitacora.log` y los datos en `bench/results/crudo.csv` y `resultados.csv`.
- Cada prueba (configuración) descarta **5 frames de calentamiento** y mide **10 frames**. Se cronometra solo el cálculo del frame (`std::chrono::steady_clock`), sin ventana (`--bench`).
- **Speedup** = $T_{v0} / T_{\text{versión}}$ usando el **promedio** de los 10 tiempos, contra la secuencial `v0` con el mismo `N` y la misma resolución. **Eficiencia** = `speedup` / hilos.
- **Fracción serial de Karp-Flatt**: $e = (1/S - 1/p) / (1 - 1/p)$.
- Los barridos A, B, C y la búsqueda de 60 FPS usan `schedule(static)` en `v2` y `v3`; el barrido D y la repetición comparan `schedules`.
- En `v3-fast` el `speedup` mezcla dos efectos: la paralelización y la vectorización de `-ffast-math`, por eso su eficiencia supera 1.

Verificación de correctitud (`--verify`)

Configuración	Schedule	Hash v0	v1	v2	v3	Suma RGB v0 = v2 = v3
N=64, 800x600, simetría 8, semilla 2026	static	e811ea536f4172b1	idéntica	idéntica	idéntica	93 422 142
N=64, 800x600	dynamic,4	e811ea536f4172b1	idéntica	idéntica	idéntica	93 422 142
N=64, 800x600	guided	e811ea536f4172b1	idéntica	idéntica	idéntica	93 422 142

Configuración	Schedule	Hash v0	v1	v2	v3	Suma RGB v0 = v2 = v3
N=16, 1280x720, simetría 12, semilla 7	static	aba0778f851b1612	idéntica	idéntica	idéntica	164 069 643
N=32, 800x600	guided	eae9492a6c34c7ae	idéntica	idéntica	idéntica	89 389 627
build -ffast-math, N=64 y N=16	static	mismos hashes que el build normal	idéntica	idéntica	idéntica	igual

En todos los casos: **0 píxeles distintos y diferencia máxima por canal 0**. Con -ffast-math los cálculos intermedios sí cambian en los últimos bits, pero en estas pruebas ninguna diferencia alcanzó a cambiar un canal de 8 bits, por lo que el hash coincidió con el del build normal.

```

== Equipo: AMD Ryzen 5 7520U with Radeon Graphics, 8 hilos lógicos
== Compilador: g++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0
== Fecha: 2026-09-23 20:30
== Verificación de correctitud
Verificacion N=64 800x600 t=3.7 hilos=8 schedule=static
v0 secuencial hash e811ea536f4172b1 suma RGB 93422142
v1 paralela hash e811ea536f4172b1 suma RGB n/a pixeles distintos 0 dif. max canal 0 -> IDENTICA
v2 paralela hash e811ea536f4172b1 suma RGB 93422142 pixeles distintos 0 dif. max canal 0 -> IDENTICA
v3 paralela hash e811ea536f4172b1 suma RGB 93422142 pixeles distintos 0 dif. max canal 0 -> IDENTICA
Verificacion N=64 800x600 t=3.7 hilos=8 schedule=dynamic:4
v0 secuencial hash e811ea536f4172b1 suma RGB 93422142
v1 paralela hash e811ea536f4172b1 suma RGB n/a pixeles distintos 0 dif. max canal 0 -> IDENTICA
v2 paralela hash e811ea536f4172b1 suma RGB 93422142 pixeles distintos 0 dif. max canal 0 -> IDENTICA
v3 paralela hash e811ea536f4172b1 suma RGB 93422142 pixeles distintos 0 dif. max canal 0 -> IDENTICA
Verificacion N=64 800x600 t=3.7 hilos=8 schedule=guided
v0 secuencial hash e811ea536f4172b1 suma RGB 93422142
v1 paralela hash e811ea536f4172b1 suma RGB n/a pixeles distintos 0 dif. max canal 0 -> IDENTICA
v2 paralela hash e811ea536f4172b1 suma RGB 93422142 pixeles distintos 0 dif. max canal 0 -> IDENTICA
v3 paralela hash e811ea536f4172b1 suma RGB 93422142 pixeles distintos 0 dif. max canal 0 -> IDENTICA
Verificacion N=16 1280x720 t=3.7 hilos=8 schedule=static
v0 secuencial hash aba0778f851b1612 suma RGB 164069643
v1 paralela hash aba0778f851b1612 suma RGB n/a pixeles distintos 0 dif. max canal 0 -> IDENTICA
v2 paralela hash aba0778f851b1612 suma RGB 164069643 pixeles distintos 0 dif. max canal 0 -> IDENTICA
v3 paralela hash aba0778f851b1612 suma RGB 164069643 pixeles distintos 0 dif. max canal 0 -> IDENTICA
-- build -ffast-math (comparar el hash de v0 con el build normal):
Verificacion N=64 800x600 t=3.7 hilos=8 schedule=static
v0 secuencial hash e811ea536f4172b1 suma RGB 93422142
v1 paralela hash e811ea536f4172b1 suma RGB n/a pixeles distintos 0 dif. max canal 0 -> IDENTICA
v2 paralela hash e811ea536f4172b1 suma RGB 93422142 pixeles distintos 0 dif. max canal 0 -> IDENTICA
v3 paralela hash e811ea536f4172b1 suma RGB 93422142 pixeles distintos 0 dif. max canal 0 -> IDENTICA

```

Resultados por barrido

Barrido A — hilos (N=64, 800x600)

versión	hilos	N	resolución	schedule	media (ms)	mediana	desv	mín	máx	FPS	speedup	eficiencia	fracción serial (Karp-Flatt)
v0	1	64	800x600	-	1444.29	1372.51	306.21	1045.25	1866.35	0.69	1.00	1.00	—

versión	hilos	N	resolución	schedule	media (ms)	mediana	desv	mín	máx	FPS	speedup	eficiencia	fracción serial (Karp-Flatt)
v1	1	64	800x600	static-collapse2	1556.40	1581.60	148.94	1293.20	1712.70	0.64	0.93	0.93	—
v1	2	64	800x600	static-collapse2	743.92	753.55	120.19	592.65	914.32	1.34	1.94	0.97	0.030
v1	4	64	800x600	static-collapse2	313.34	310.19	13.75	297.90	336.53	3.19	4.61	1.15	-0.044
v1	6	64	800x600	static-collapse2	288.59	283.69	31.37	248.46	330.99	3.46	5.00	0.83	0.040
v1	8	64	800x600	static-collapse2	320.71	287.74	75.96	235.84	459.10	3.12	4.50	0.56	0.111
v2	1	64	800x600	static	1591.26	1632.24	197.56	1240.50	1851.64	0.63	0.91	0.91	—
v2	2	64	800x600	static	625.27	622.60	60.65	554.07	735.96	1.60	2.31	1.15	-0.134
v2	4	64	800x600	static	370.75	367.06	33.64	319.73	434.34	2.70	3.90	0.97	0.009
v2	6	64	800x600	static	281.72	269.70	34.21	259.39	374.98	3.55	5.13	0.85	0.034
v2	8	64	800x600	static	238.26	236.70	7.88	228.16	255.67	4.20	6.06	0.76	0.046
v3	1	64	800x600	static	1390.52	1376.53	192.28	1134.84	1708.54	0.72	1.04	1.04	—
v3	2	64	800x600	static	688.94	657.77	163.05	538.63	1090.96	1.45	2.10	1.05	-0.046
v3	4	64	800x600	static	362.25	367.11	11.41	337.35	373.34	2.76	3.99	1.00	0.001
v3	6	64	800x600	static	264.70	257.86	22.27	236.32	309.75	3.78	5.46	0.91	0.020
v3	8	64	800x600	static	248.55	248.20	39.80	201.95	314.02	4.02	5.81	0.73	0.054
v3-fast	1	64	800x600	static	327.80	322.77	26.66	299.62	373.52	3.05	4.41	4.41	—
v3-fast	2	64	800x600	static	204.87	206.40	15.91	180.87	225.94	4.88	7.05	3.52	-0.716
v3-fast	4	64	800x600	static	112.84	113.07	3.67	106.74	118.05	8.86	12.80	3.20	-0.229
v3-fast	6	64	800x600	static	101.25	95.54	18.02	91.53	151.90	9.88	14.26	2.38	-0.116
v3-fast	8	64	800x600	static	138.56	116.00	48.64	98.13	223.57	7.22	10.42	1.30	-0.033

Barrido B — N (800x600, 8 hilos)

versión	hilos	N	resolución	schedule	media (ms)	mediana	desv	mín	máx	FPS	speedup	eficiencia
v0	1	4	800x600	-	241.96	251.70	21.89	206.21	267.91	4.13	1.00	1.00
v1	8	4	800x600	static-collapse2	53.23	51.77	6.38	46.39	70.22	18.79	4.55	0.57
v2	8	4	800x600	static	45.14	45.77	2.14	42.46	48.73	22.16	5.36	0.67
v3	8	4	800x600	static	35.92	35.93	3.71	31.19	41.53	27.84	6.74	0.84
v3-fast	8	4	800x600	static	26.00	24.45	4.42	21.41	35.69	38.47	9.31	1.16
v0	1	16	800x600	-	536.37	538.26	118.97	372.46	752.40	1.86	1.00	1.00
v1	8	16	800x600	static-collapse2	102.06	102.53	7.82	90.19	114.39	9.80	5.26	0.66
v2	8	16	800x600	static	97.57	100.93	17.94	73.51	121.89	10.25	5.50	0.69
v3	8	16	800x600	static	65.20	62.48	7.16	59.15	84.28	15.34	8.23	1.03
v3-fast	8	16	800x600	static	30.32	29.47	2.98	26.67	36.29	32.98	17.69	2.21
v0	1	64	800x600	-	1444.29	1372.51	306.21	1045.25	1866.35	0.69	1.00	1.00
v1	8	64	800x600	static-collapse2	320.71	287.74	75.96	235.84	459.10	3.12	4.50	0.56
v2	8	64	800x600	static	238.26	236.70	7.88	228.16	255.67	4.20	6.06	0.76
v3	8	64	800x600	static	248.55	248.20	39.80	201.95	314.02	4.02	5.81	0.73
v3-fast	8	64	800x600	static	138.56	116.00	48.64	98.13	223.57	7.22	10.42	1.30
v0	1	256	800x600	-	6697.97	7000.03	702.55	5484.54	7346.11	0.15	1.00	1.00
v1	8	256	800x600	static-collapse2	1074.69	977.22	214.66	857.17	1541.38	0.93	6.23	0.78
v2	8	256	800x600	static	994.86	985.81	42.82	935.65	1063.98	1.00	6.73	0.84
v3	8	256	800x600	static	1030.71	1031.44	85.04	896.84	1167.18	0.97	6.50	0.81
v3-fast	8	256	800x600	static	406.42	368.58	111.06	322.03	691.41	2.46	16.48	2.06

versión	hilos	N	resolución	schedule	media (ms)	mediana	desv	mín	máx	FPS	speedup	eficiencia
v0	1	1024	800x600	-	28979.38	28857.30	926.11	27406.52	30533.77	0.04	1.00	1.00
v1	8	1024	800x600	static-collapse2	2861.89	2835.49	214.67	2601.51	3387.91	0.35	10.13	1.27
v2	8	1024	800x600	static	3374.97	3302.08	347.81	2929.77	3986.31	0.30	8.59	1.07
v3	8	1024	800x600	static	3541.42	3447.33	551.70	2918.14	4478.24	0.28	8.18	1.02
v3-fast	8	1024	800x600	static	1353.80	1338.66	219.46	1018.93	1678.92	0.74	21.41	2.68

Barrido C — resolución (N=64, 8 hilos)

versión	hilos	N	resolución	schedule	media (ms)	mediana	desv	mín	máx	FPS	speedup	eficiencia
v0	1	64	640x480	-	1057.21	1066.02	165.14	781.10	1290.16	0.95	1.00	1.00
v1	8	64	640x480	static-collapse2	134.48	131.81	11.94	119.21	155.64	7.44	7.86	0.98
v2	8	64	640x480	static	144.27	138.82	12.75	132.65	173.12	6.93	7.33	0.92
v3	8	64	640x480	static	126.18	123.54	7.46	114.89	141.53	7.92	8.38	1.05
v3-fast	8	64	640x480	static	60.73	55.06	14.84	46.29	87.04	16.46	17.41	2.18
v0	1	64	800x600	-	1444.29	1372.51	306.21	1045.25	1866.35	0.69	1.00	1.00
v1	8	64	800x600	static-collapse2	320.71	287.74	75.96	235.84	459.10	3.12	4.50	0.56
v2	8	64	800x600	static	238.26	236.70	7.88	228.16	255.67	4.20	6.06	0.76
v3	8	64	800x600	static	248.55	248.20	39.80	201.95	314.02	4.02	5.81	0.73
v3-fast	8	64	800x600	static	138.56	116.00	48.64	98.13	223.57	7.22	10.42	1.30
v0	1	64	1280x720	-	2815.35	2716.38	290.79	2513.90	3313.79	0.35	1.00	1.00
v1	8	64	1280x720	static-collapse2	614.53	599.61	75.20	515.71	747.20	1.63	4.58	0.57
v2	8	64	1280x720	static	502.85	505.27	62.43	412.42	610.89	1.99	5.60	0.70
v3	8	64	1280x720	static	401.16	395.15	33.49	360.54	463.61	2.49	7.02	0.88
v3-fast	8	64	1280x720	static	164.83	165.76	7.95	151.61	173.91	6.07	17.08	2.14
v0	1	64	1920x1080	-	6841.70	6906.00	620.27	5474.40	7615.34	0.15	1.00	1.00
v1	8	64	1920x1080	static-collapse2	939.65	928.42	105.02	824.46	1196.73	1.06	7.28	0.91
v2	8	64	1920x1080	static	884.24	869.78	92.48	798.12	1123.00	1.13	7.74	0.97
v3	8	64	1920x1080	static	865.19	850.83	55.17	791.26	959.89	1.16	7.91	0.99
v3-fast	8	64	1920x1080	static	344.33	340.16	35.75	308.20	415.27	2.90	19.87	2.48

Barrido D — schedules (N=64, 800x600, 8 hilos)

versión	hilos	N	resolución	schedule	media (ms)	mediana	desv	mín	máx	FPS	speedup	eficiencia
v1	8	64	800x600	static-collapse2	320.71	287.74	75.96	235.84	459.10	3.12	4.50	0.56
v2	8	64	800x600	static	238.26	236.70	7.88	228.16	255.67	4.20	6.06	0.76
v3	8	64	800x600	static	248.55	248.20	39.80	201.95	314.02	4.02	5.81	0.73
v2	8	64	800x600	static:8	231.37	223.51	22.11	202.33	264.58	4.32	6.24	0.78
v2	8	64	800x600	dynamic:1	199.36	193.38	18.07	181.81	234.51	5.02	7.24	0.91
v2	8	64	800x600	dynamic:4	232.59	230.86	33.95	189.18	307.43	4.30	6.21	0.78
v2	8	64	800x600	dynamic:16	213.18	214.24	20.33	186.85	245.92	4.69	6.77	0.85
v2	8	64	800x600	guided	198.15	190.33	21.54	177.83	240.01	5.05	7.29	0.91
v3	8	64	800x600	static:8	223.92	226.92	17.11	198.34	249.04	4.47	6.45	0.81
v3	8	64	800x600	dynamic:1	214.43	213.25	28.73	172.13	260.88	4.66	6.74	0.84
v3	8	64	800x600	dynamic:4	210.27	210.09	20.38	183.84	242.18	4.76	6.87	0.86
v3	8	64	800x600	dynamic:16	190.57	191.65	7.31	177.19	201.12	5.25	7.58	0.95
v3	8	64	800x600	guided	177.51	173.45	10.06	170.07	196.78	5.63	8.14	1.02

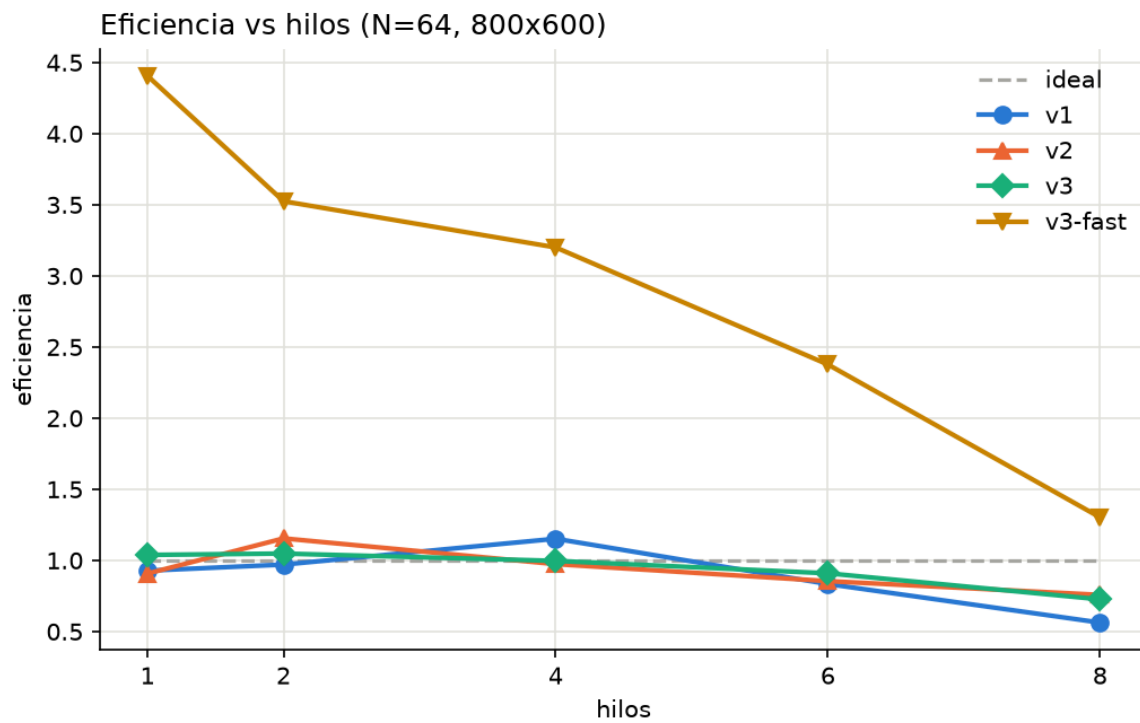
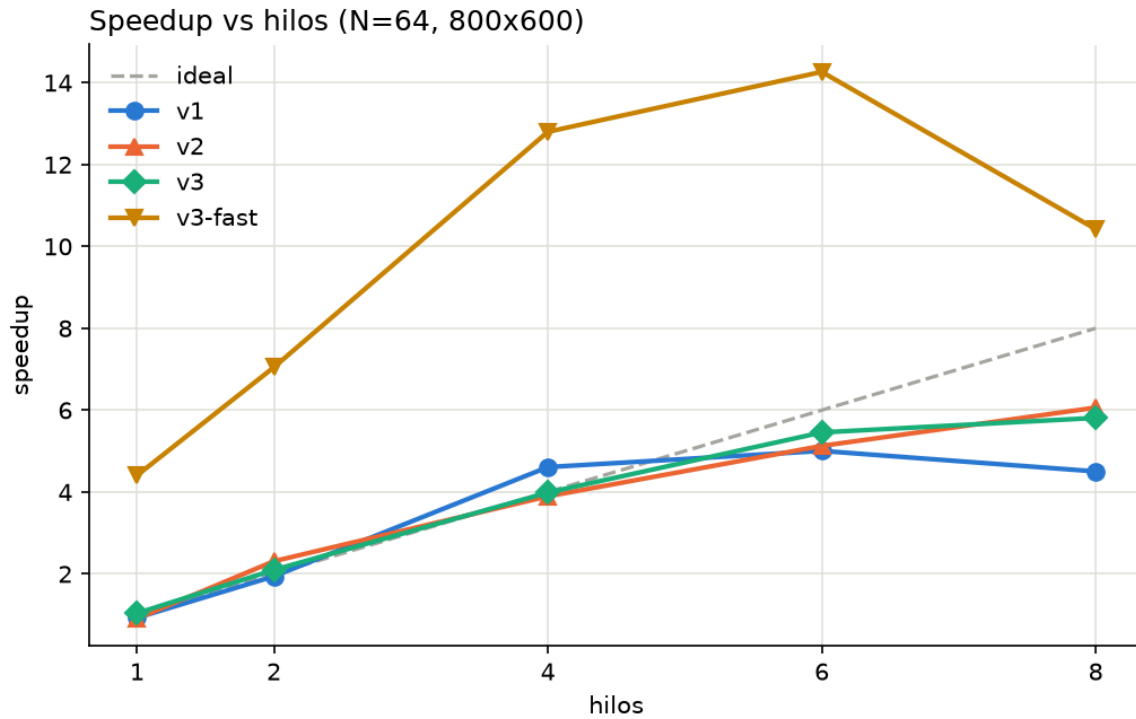
N máximo que sostiene 60 FPS (640x480)

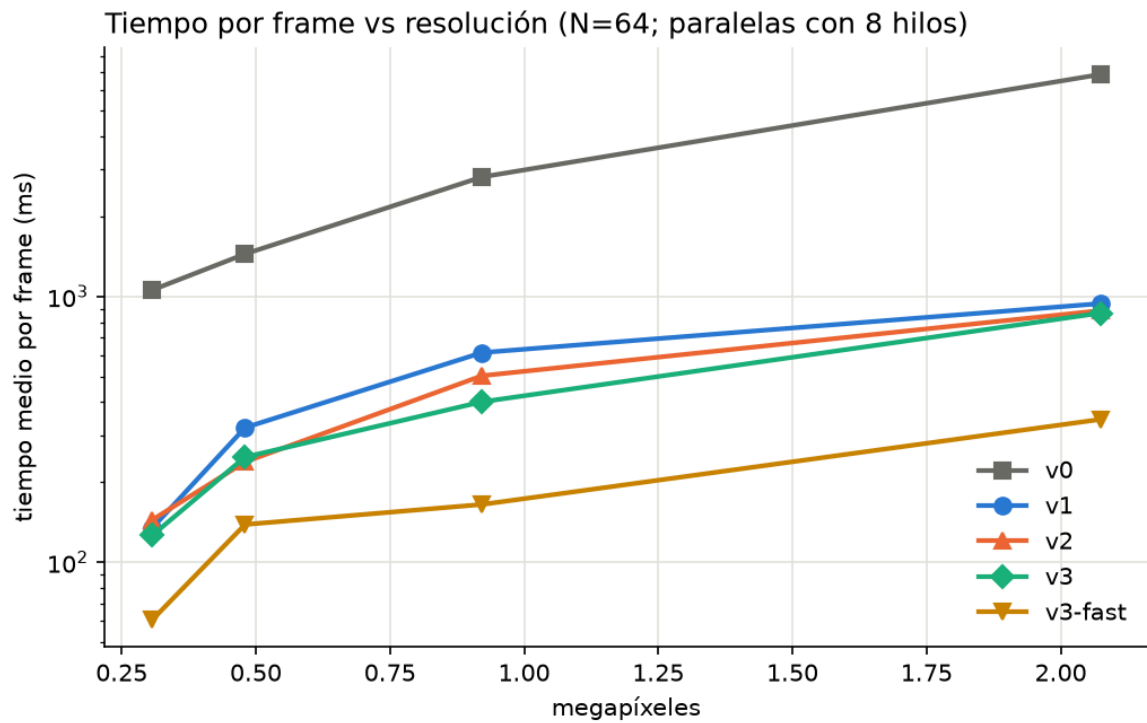
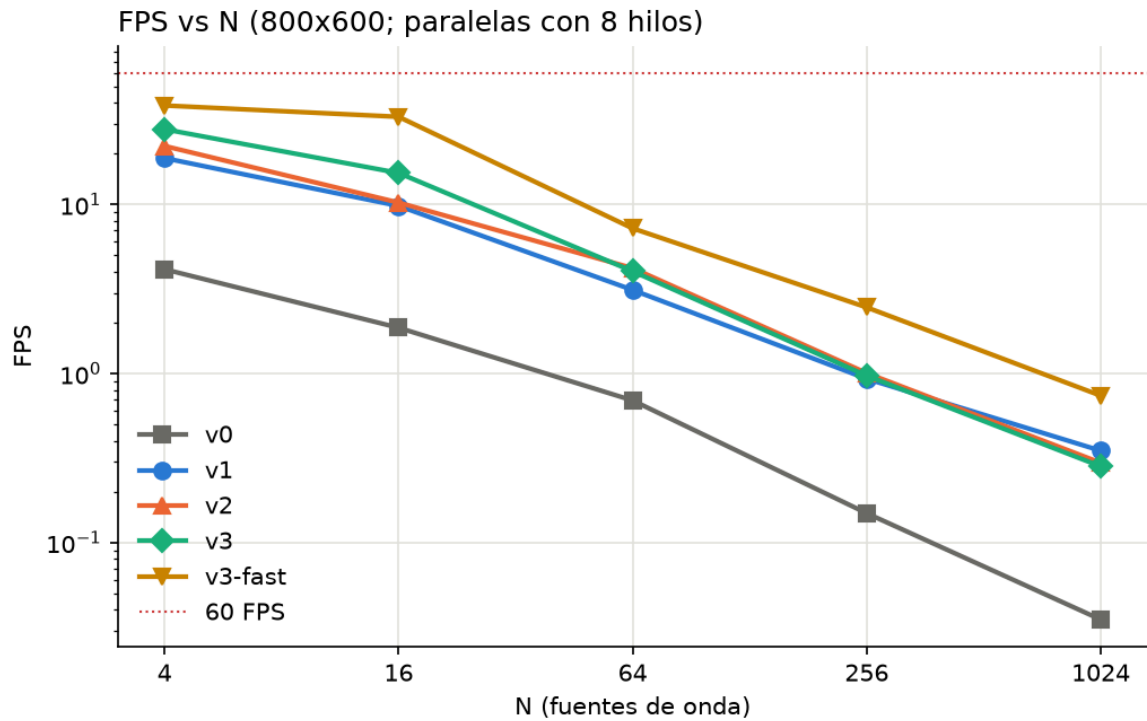
versión	N máximo	FPS con ese N	FPS con N=1
v0	no alcanza	0	7.87
v1	no alcanza	0	58.74
v2	no alcanza	0	56.73
v3	3	69.45	91.26
v3-fast	16	60.69	117.26

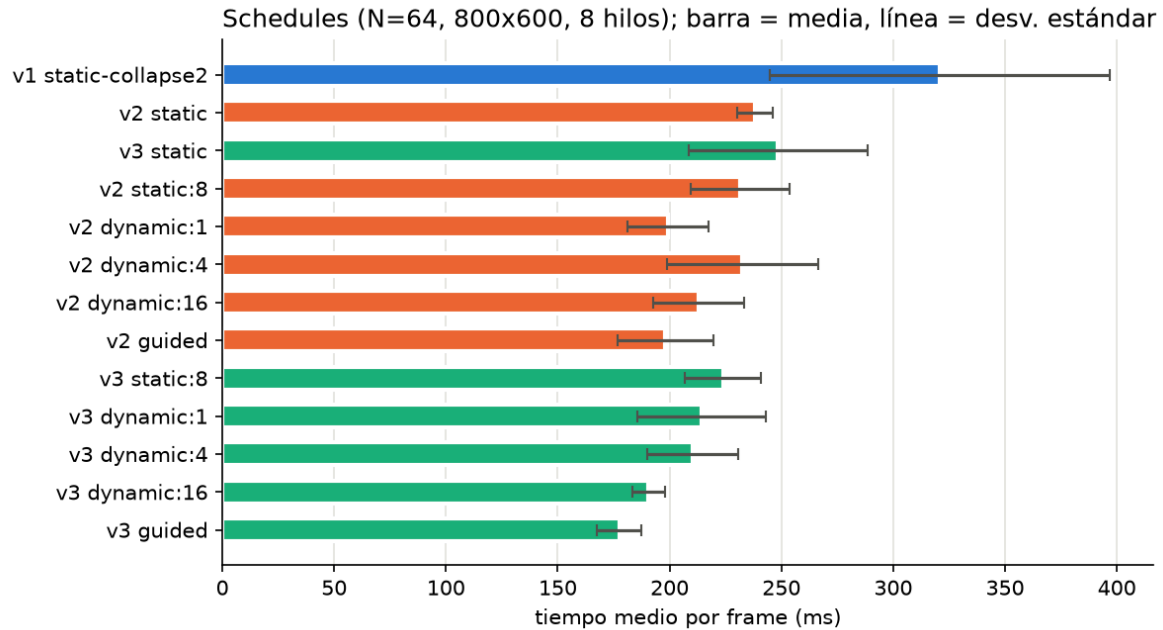
Repetición intercalada (N=64, 800x600, 8 hilos, 3 rondas de 10 mediciones)

versión	schedule	media ronda 1 (ms)	ronda 2	ronda 3	promedio	speedup
v3	static	198.7	207.7	191.5	199.3	7.81
v3	static:8	207.4	207.3	197.3	204.0	7.63
v3	dynamic:4	189.9	171.7	193.2	185.0	8.41
v3	dynamic:16	199.1	198.6	184.0	193.9	8.02
v3	guided	189.0	201.2	172.3	187.5	8.30
v1	static-collapse2	245.6	202.0	210.0	219.2	7.10
v0	-	1446.4	1555.2	1665.2	1555.6	1.00

Gráficas







Capturas de las mediciones

Barrido A en consola (cada línea es una prueba; debajo, sus 10 mediciones):

```
== Barrido A: hilos (N=64, 800x600)
[bench] v0 hilos=1 schedule=- N=64 800x600 | media 1444.286 ms | mediana 1372.512 | desv 306.207 | min 1045.255 | max 1866.346 | 0.69 FPS
muestras (ms): 1293.748 1209.415 1451.275 1260.753 1125.296 1045.255 1559.919 1866.346 1768.918 1861.939
[bench] v1 hilos=1 schedule=static-collapse2 N=64 800x600 | media 1556.399 ms | mediana 1581.600 | desv 148.935 | min 1293.199 | max 1712.697 | 0.64 FPS
muestras (ms): 1667.246 1647.421 1708.821 1515.779 1506.053 1293.199 1390.601 1712.697 1684.395 1437.780
[bench] v2 hilos=1 schedule=static N=64 800x600 | media 1591.261 ms | mediana 1632.240 | desv 197.564 | min 1240.503 | max 1851.639 | 0.63 FPS
muestras (ms): 1673.722 1521.835 1590.757 1851.639 1734.394 1770.925 1698.877 1520.666 1240.503 1309.294
[bench] v3 hilos=1 schedule=static N=64 800x600 | media 1390.517 ms | mediana 1376.526 | desv 192.283 | min 1134.839 | max 1708.544 | 0.72 FPS
muestras (ms): 1373.718 1153.740 1134.839 1583.506 1708.544 1379.334 1229.559 1356.662 1603.577 1381.689
[bench] v3-fast hilos=1 schedule=static N=64 800x600 | media 327.796 ms | mediana 322.772 | desv 26.658 | min 299.624 | max 373.519 | 3.05 FPS
muestras (ms): 340.558 335.943 301.482 309.906 310.247 299.624 305.852 365.535 373.519 335.297
[bench] v1 hilos=2 schedule=static-collapse2 N=64 800x600 | media 743.915 ms | mediana 753.554 | desv 120.187 | min 592.648 | max 914.323 | 1.34 FPS
muestras (ms): 627.207 815.210 914.323 824.795 736.903 651.045 592.648 900.536 770.206 606.279
[bench] v2 hilos=2 schedule=static N=64 800x600 | media 625.273 ms | mediana 622.598 | desv 60.652 | min 554.070 | max 735.961 | 1.60 FPS
muestras (ms): 593.254 554.070 639.204 667.631 735.961 605.992 686.857 646.807 563.075 559.878
[bench] v3 hilos=2 schedule=static N=64 800x600 | media 688.935 ms | mediana 657.768 | desv 163.052 | min 538.627 | max 1090.963 | 1.45 FPS
muestras (ms): 639.763 795.715 1090.963 724.725 675.774 694.988 559.025 609.527 560.248 538.627
[bench] v3-fast hilos=2 schedule=static N=64 800x600 | media 204.872 ms | mediana 206.398 | desv 15.907 | min 180.868 | max 225.940 | 4.88 FPS
muestras (ms): 210.131 189.570 187.392 180.868 197.350 207.762 219.604 205.035 225.940 225.069
[bench] v1 hilos=4 schedule=static-collapse2 N=64 800x600 | media 313.339 ms | mediana 310.190 | desv 13.753 | min 297.905 | max 336.529 | 3.19 FPS
muestras (ms): 336.529 305.777 304.382 297.905 298.838 310.208 310.172 315.418 317.682 336.480
[bench] v2 hilos=4 schedule=static N=64 800x600 | media 370.751 ms | mediana 367.060 | desv 33.643 | min 319.730 | max 434.344 | 2.70 FPS
muestras (ms): 346.125 369.963 338.814 319.730 361.839 371.301 364.157 399.041 402.193 434.344
```

Búsqueda del N máximo a 60 FPS (640x480):

```

== Búsqueda de N máximo a 60 FPS (640x480)
[bench] v0 hilos=1 schedule=- N=1 640x480 | media 127.067 ms | mediana 126.761 | desv 18.069 | min 100.831 | max 152.209 | 7.87 FPS
v0,0,0,7.87
[bench] v1 hilos=8 schedule=static-collapse2 N=1 640x480 | media 17.025 ms | mediana 16.811 | desv 0.979 | min 15.751 | max 18.823 | 58.74 FPS
v1,0,0,58.74
[bench] v2 hilos=8 schedule=static N=1 640x480 | media 17.628 ms | mediana 17.706 | desv 1.134 | min 15.985 | max 20.083 | 56.73 FPS
v2,0,0,56.73
[bench] v3 hilos=8 schedule=static N=1 640x480 | media 10.957 ms | mediana 10.912 | desv 0.477 | min 10.427 | max 11.905 | 91.26 FPS
[bench] v3 hilos=8 schedule=static N=2 640x480 | media 12.567 ms | mediana 12.651 | desv 0.862 | min 10.950 | max 13.950 | 79.58 FPS
[bench] v3 hilos=8 schedule=static N=4 640x480 | media 16.797 ms | mediana 16.671 | desv 1.894 | min 14.694 | max 20.646 | 59.53 FPS
[bench] v3 hilos=8 schedule=static N=3 640x480 | media 14.400 ms | mediana 14.353 | desv 0.911 | min 12.536 | max 15.746 | 69.45 FPS
v3,3,69.45,91.26
[bench] v3-fast hilos=8 schedule=static N=1 640x480 | media 8.528 ms | mediana 8.480 | desv 0.399 | min 8.060 | max 9.446 | 117.26 FPS
[bench] v3-fast hilos=8 schedule=static N=2 640x480 | media 9.227 ms | mediana 8.831 | desv 1.125 | min 7.893 | max 11.667 | 108.38 FPS
[bench] v3-fast hilos=8 schedule=static N=4 640x480 | media 9.596 ms | mediana 9.701 | desv 0.387 | min 9.061 | max 10.112 | 104.21 FPS
[bench] v3-fast hilos=8 schedule=static N=8 640x480 | media 12.266 ms | mediana 11.610 | desv 1.140 | min 11.288 | max 14.592 | 81.53 FPS
[bench] v3-fast hilos=8 schedule=static N=16 640x480 | media 16.476 ms | mediana 16.209 | desv 0.740 | min 15.743 | max 17.977 | 60.69 FPS
[bench] v3-fast hilos=8 schedule=static N=32 640x480 | media 40.529 ms | mediana 39.379 | desv 4.342 | min 35.055 | max 46.398 | 24.67 FPS
[bench] v3-fast hilos=8 schedule=static N=24 640x480 | media 32.315 ms | mediana 30.222 | desv 4.595 | min 27.048 | max 39.547 | 30.95 FPS
[bench] v3-fast hilos=8 schedule=static N=20 640x480 | media 27.551 ms | mediana 27.528 | desv 4.231 | min 19.797 | max 32.073 | 36.30 FPS
[bench] v3-fast hilos=8 schedule=static N=18 640x480 | media 18.382 ms | mediana 18.125 | desv 1.550 | min 17.033 | max 22.485 | 54.40 FPS
[bench] v3-fast hilos=8 schedule=static N=17 640x480 | media 29.947 ms | mediana 24.250 | desv 10.220 | min 21.549 | max 52.024 | 33.39 FPS
v3-fast,16,60.69,117.26
== Listo: bench/results/crudo.csv, bench/results/bitacora.log

```

Ejecución interactiva (bajo xvfb-run; el FPS incluye la presentación):

```

$ ./screensaver_seq --n 14 --frames 12 --captura capturas/seq_v0_n14.bmp
Captura guardada en capturas/seq_v0_n14.bmp
12 frames en 5.92 s -> 2.03 FPS promedio (FPS 2.9 | N=14 | 800x600 | v0 | 1 hilo | calculo 363.6 ms)

$ ./screensaver_par --n 14 --version 3 --frames 80 --captura capturas/screensaver_v3_n14.bmp
Captura guardada en capturas/screensaver_v3_n14.bmp
80 frames en 6.55 s -> 12.21 FPS promedio (FPS 13.2 | N=14 | 800x600 | v3 | 8 hilos | calculo 59.6 ms)

$ ./screensaver_par --n 14 --version 1 --frames 60 --captura capturas/par_v1_n14.bmp
Captura guardada en capturas/par_v1_n14.bmp
60 frames en 5.11 s -> 11.74 FPS promedio (FPS 14.5 | N=14 | 800x600 | v1 | 8 hilos | calculo 60.3 ms)

$ ./screensaver_par --n 40 --ancho 640 --alto 480 --simetria 12 --version 2 --frames 60 --captura capturas/par_v2_n40_sim12.bmp
Captura guardada en capturas/par_v2_n40_sim12.bmp
60 frames en 5.58 s -> 10.75 FPS promedio (FPS 11.8 | N=40 | 640x480 | v2 | 8 hilos | calculo 74.5 ms)

$ ./screensaver_par --n 64 --ancho 1280 --alto 720 --simetria 6 --semilla 99 --frames 30 --captura capturas/par_v3_n64_1280.bmp
Captura guardada en capturas/par_v3_n64_1280.bmp
30 frames en 11.42 s -> 2.63 FPS promedio (FPS 2.8 | N=64 | 1280x720 | v3 | 8 hilos | calculo 384.6 ms)

$ ./screensaver_par_fast --n 16 --ancho 640 --alto 480 --frames 200 --captura capturas/par_v3fast_n16_640.bmp
Captura guardada en capturas/par_v3fast_n16_640.bmp
200 frames en 4.78 s -> 41.81 FPS promedio (FPS 35.6 | N=16 | 640x480 | v3-fast | 8 hilos | calculo 20.7 ms)

```

Programación defensiva: respuesta del programa ante argumentos inválidos (todos terminan con código 1, sin fallar):

```

$ ./screensaver_par
Error: --n es obligatorio.
Uso: ./screensaver_par --n <N> [--ancho W] [--alto H] [--simetria S] [--semilla X]
(código de salida: 1)

$ ./screensaver_par --n 0
Error: --n = 0 fuera de rango [1, 4096].
Uso: ./screensaver_par --n <N> [--ancho W] [--alto H] [--simetria S] [--semilla X]
(código de salida: 1)

$ ./screensaver_par --n -5
Error: --n = -5 fuera de rango [1, 4096].
Uso: ./screensaver_par --n <N> [--ancho W] [--alto H] [--simetria S] [--semilla X]
(código de salida: 1)

$ ./screensaver_par --n 10 --ancho 12abc
Error: --ancho debe ser un entero, se recibió "12abc".
Uso: ./screensaver_par --n <N> [--ancho W] [--alto H] [--simetria S] [--semilla X]
(código de salida: 1)

$ ./screensaver_par --n 10 --alto 300
Error: --alto = 300 fuera de rango [480, 2160].
Uso: ./screensaver_par --n <N> [--ancho W] [--alto H] [--simetria S] [--semilla X]
(código de salida: 1)

$ ./screensaver_par --n 10 --colores 5
Error: argumento desconocido "--colores".
Uso: ./screensaver_par --n <N> [--ancho W] [--alto H] [--simetria S] [--semilla X]
(código de salida: 1)

$ ./screensaver_par --n 9999999999
Error: --n = 9999999999 fuera de rango [1, 4096].
Uso: ./screensaver_par --n <N> [--ancho W] [--alto H] [--simetria S] [--semilla X]
(código de salida: 1)

$ ./screensaver_par --n 10 --bench --frames 3
Error: --bench requiere al menos 10 frames medidos.
(código de salida: 1)

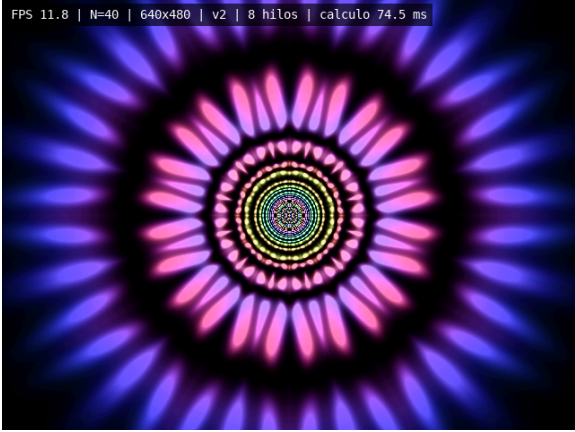
$ ./screensaver_par --n 10 --version 4
Error: --version = 4 fuera de rango [1, 3].
Uso: ./screensaver_par --n <N> [--ancho W] [--alto H] [--simetria S] [--semilla X]
(código de salida: 1)

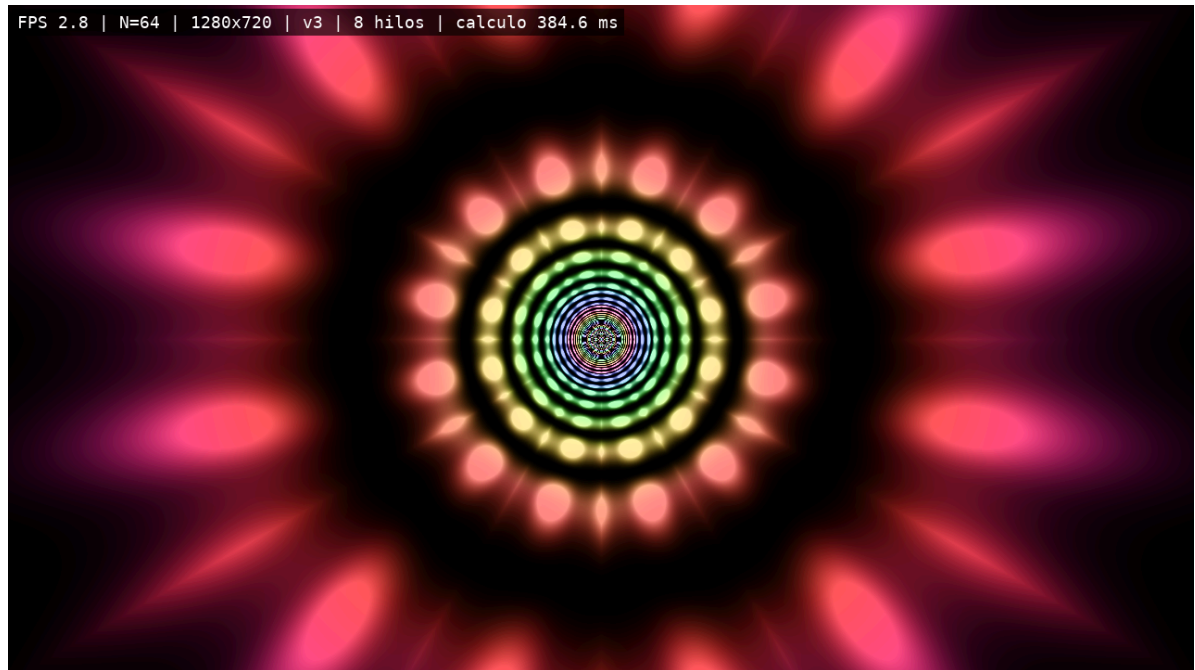
$ ./screensaver_par --n 10 --schedule auto
Error: --schedule debe ser static, dynamic o guided.
Uso: ./screensaver_par --n <N> [--ancho W] [--alto H] [--simetria S] [--semilla X]
(código de salida: 1)

$ ./screensaver_seq --n 10 --hilos 4
Error: argumento desconocido "--hilos".

```

Capturas del screensaver

 FPS 2.9 N=14 800x600 v0 1 hilo calculo 363.6 ms	 FPS 13.2 N=14 800x600 v3 8 hilos calculo 59.6 ms
v0 secuencial, N=14, 800x600	v3, 8 hilos, N=14, 800x600
 FPS 11.8 N=40 640x480 v2 8 hilos calculo 74.5 ms	 FPS 35.6 N=16 640x480 v3-fast 8 hilos calculo 20.7 ms
v2, N=40, 640x480, simetría 12	v3-fast, N=16, 640x480



v3, N=64, 1280x720, simetría 6, semilla 99.

Mediciones individuales

Las 10 mediciones (ms) de cada una de las 66 pruebas, en el orden en que se ejecutaron.

#	versión	hilos	N	resolución	schedule	m1	m2	m3	m4	m5	m6	m7	m8	m9	m10
1	v0	1	64	800x600	-	1293.7	1209.4	1451.3	1260.8	1125.3	1045.3	1559.9	1866.3	1768.9	1861.9
2	v1	1	64	800x600	static-collapse2	1667.2	1647.4	1708.8	1515.8	1506.1	1293.2	1390.6	1712.7	1684.4	1437.8
3	v2	1	64	800x600	static	1673.7	1521.8	1590.8	1851.6	1734.4	1770.9	1698.9	1520.7	1240.5	1309.3
4	v3	1	64	800x600	static	1373.7	1153.7	1134.8	1583.5	1708.5	1379.3	1229.6	1356.7	1603.6	1381.7
5	v3-fast	1	64	800x600	static	340.6	335.9	301.5	309.9	310.2	299.6	305.9	365.5	373.5	335.3
6	v1	2	64	800x600	static-collapse2	627.2	815.2	914.3	824.8	736.9	651.0	592.6	900.5	770.2	606.3
7	v2	2	64	800x600	static	593.3	554.1	639.2	667.6	736.0	606.0	686.9	646.8	563.1	559.9
8	v3	2	64	800x600	static	639.8	795.7	1091.0	724.7	675.8	695.0	559.0	609.5	560.2	538.6
9	v3-fast	2	64	800x600	static	210.1	189.6	187.4	180.9	197.4	207.8	219.6	205.0	225.9	225.1
10	v1	4	64	800x600	static-collapse2	336.5	305.8	304.4	297.9	298.8	310.2	310.2	315.4	317.7	336.5
11	v2	4	64	800x600	static	346.1	370.0	338.8	319.7	361.8	371.3	364.2	399.0	402.2	434.3
12	v3	4	64	800x600	static	366.1	355.3	368.1	370.3	370.0	349.7	369.2	337.4	363.1	373.3
13	v3-fast	4	64	800x600	static	118.1	108.3	113.3	112.9	106.7	109.9	112.8	113.6	115.7	117.1
14	v1	6	64	800x600	static-collapse2	248.5	288.5	278.9	326.2	302.3	274.1	325.1	331.0	250.8	260.7
15	v2	6	64	800x600	static	285.8	261.9	269.0	261.7	283.1	259.4	270.4	375.0	284.2	266.7
16	v3	6	64	800x600	static	254.6	254.8	309.7	260.9	267.6	236.3	250.8	244.6	277.9	289.9
17	v3-fast	6	64	800x600	static	97.6	94.8	101.6	94.7	96.3	97.2	91.5	92.7	94.1	151.9
18	v1	8	64	800x600	static-collapse2	271.2	285.8	416.8	459.1	402.7	289.6	283.4	297.0	235.8	265.6
19	v2	8	64	800x600	static	243.0	241.8	242.1	228.2	232.8	230.6	236.7	235.0	236.7	255.7
20	v3	8	64	800x600	static	314.0	214.5	202.0	202.2	215.7	295.0	273.4	254.2	242.2	272.3
21	v3-fast	8	64	800x600	static	218.7	223.6	173.9	127.0	111.7	107.5	98.1	101.4	103.5	120.3

#	versión	hilos	N	resolución	schedule	m1	m2	m3	m4	m5	m6	m7	m8	m9	m10
22	v0	1	4	800x600	-	206.2	214.2	228.1	223.8	247.9	267.9	256.2	257.9	255.5	261.9
23	v1	8	4	800x600	static-collapse2	51.5	53.7	46.4	70.2	54.3	52.9	51.2	52.0	50.3	49.7
24	v2	8	4	800x600	static	46.6	42.5	46.3	44.0	46.6	48.7	46.4	42.7	42.5	45.2
25	v3	8	4	800x600	static	37.3	41.5	37.9	41.0	32.0	37.8	31.2	34.5	32.5	33.4
26	v3-fast	8	4	800x600	static	23.2	23.7	21.4	25.6	25.0	31.4	35.7	27.4	23.9	22.8
27	v0	1	16	800x600	-	752.4	592.1	537.9	538.6	411.2	645.3	487.8	372.5	612.4	413.6
28	v1	8	16	800x600	static-collapse2	102.8	111.7	107.1	100.1	95.4	104.0	102.2	90.2	92.7	114.4
29	v2	8	16	800x600	static	110.6	121.9	111.3	112.6	77.2	73.5	91.2	112.3	85.6	79.4
30	v3	8	16	800x600	static	66.0	62.1	68.4	62.2	61.7	62.8	84.3	63.3	59.1	62.1
31	v3-fast	8	16	800x600	static	26.7	28.7	27.9	27.7	29.5	29.5	36.3	32.6	33.0	31.5
32	v0	1	256	800x600	-	5484.5	5711.5	6015.9	7242.8	7203.8	7251.7	6723.3	7185.6	7346.1	6814.5
33	v1	8	256	800x600	static-collapse2	857.2	962.5	989.2	1541.4	1328.0	987.1	967.4	1206.0	947.9	960.4
34	v2	8	256	800x600	static	969.5	1064.0	986.5	985.1	966.2	1005.3	935.7	1045.2	1040.0	951.1
35	v3	8	256	800x600	static	1038.9	1018.6	1079.1	1048.4	1139.2	945.0	1167.2	896.8	950.0	1024.0
36	v3-fast	8	256	800x600	static	691.4	362.9	490.9	374.3	358.7	322.0	342.3	405.4	386.0	330.3
37	v0	1	1024	800x600	-	27406.5	28783.7	28345.0	28315.7	28620.4	29132.4	28930.9	30203.7	30533.8	29521.8
38	v1	8	1024	800x600	static-collapse2	2601.5	2670.5	2953.3	3387.9	2872.3	2894.3	2905.2	2752.5	2782.6	2798.7
39	v2	8	1024	800x600	static	2929.8	2975.7	3159.1	3693.0	3475.8	3986.3	3173.3	3752.6	3249.5	3354.7
40	v3	8	1024	800x600	static	2986.7	2918.1	3156.0	3690.0	4201.7	4033.9	4478.2	3054.9	3271.2	3623.4
41	v3-fast	8	1024	800x600	static	1018.9	1389.9	1162.5	1239.9	1163.5	1316.6	1360.8	1638.1	1678.9	1569.0
42	v0	1	64	640x480	-	781.1	1062.5	1164.5	1069.5	1290.2	1248.0	1120.6	1056.3	831.7	947.8
43	v1	8	64	640x480	static-collapse2	122.9	131.7	139.3	125.7	148.7	155.6	131.9	119.2	126.2	143.5
44	v2	8	64	640x480	static	173.1	138.6	132.7	157.0	137.4	147.2	148.9	134.0	134.8	139.0
45	v3	8	64	640x480	static	141.5	122.5	130.8	122.2	114.9	123.0	128.9	132.6	121.2	124.1
46	v3-fast	8	64	640x480	static	50.3	87.0	87.0	61.1	56.6	64.0	52.2	46.3	49.3	53.5
47	v0	1	64	1280x720	-	3143.8	2622.7	3064.2	2541.5	2998.3	3313.8	2693.4	2513.9	2739.3	2522.6
48	v1	8	64	1280x720	static-collapse2	549.8	563.0	605.8	655.1	515.7	593.4	607.2	578.7	729.4	747.2
49	v2	8	64	1280x720	static	562.1	553.8	471.4	610.9	502.6	510.3	415.8	481.3	412.4	507.9
50	v3	8	64	1280x720	static	393.5	431.4	399.9	439.3	374.2	384.0	463.6	368.3	396.8	360.5
51	v3-fast	8	64	1280x720	static	167.8	156.1	163.7	169.4	173.3	173.9	172.8	151.6	156.8	162.8
52	v0	1	64	1920x1080	-	5474.4	7007.1	6231.9	6715.5	7184.1	6804.9	6779.5	7173.5	7615.3	7430.7
53	v1	8	64	1920x1080	static-collapse2	864.4	1196.7	932.5	976.5	849.0	891.9	983.5	824.5	924.4	953.1
54	v2	8	64	1920x1080	static	916.3	824.9	889.6	865.4	817.3	874.1	831.9	798.1	1123.0	901.7
55	v3	8	64	1920x1080	static	812.6	821.2	791.3	885.7	845.3	848.2	853.5	945.5	888.9	959.9
56	v3-fast	8	64	1920x1080	static	310.2	346.9	308.2	321.2	358.2	310.1	388.2	351.6	333.4	415.3
57	v2	8	64	800x600	static:8	207.6	222.3	219.5	202.3	224.7	252.9	257.7	246.4	215.8	264.6
58	v2	8	64	800x600	dynamic:1	182.9	187.9	194.6	192.2	197.6	181.8	185.5	217.0	219.7	234.5
59	v2	8	64	800x600	dynamic:4	232.1	233.7	229.6	262.6	307.4	214.8	189.2	217.5	242.7	196.2
60	v2	8	64	800x600	dynamic:16	235.7	245.9	232.0	195.4	196.2	219.1	211.9	192.2	186.9	216.6
61	v2	8	64	800x600	guided	179.2	181.6	192.5	240.0	202.0	177.8	188.2	224.9	213.8	181.6
62	v3	8	64	800x600	static:8	225.6	218.6	228.2	235.5	232.8	202.5	249.0	198.3	206.6	242.1
63	v3	8	64	800x600	dynamic:1	260.9	243.0	235.0	200.5	221.0	205.5	228.1	174.0	172.1	204.1
64	v3	8	64	800x600	dynamic:4	190.4	183.8	189.7	229.5	242.2	235.0	197.5	214.5	205.8	214.4
65	v3	8	64	800x600	dynamic:16	184.3	195.7	187.7	193.3	192.2	201.1	198.8	184.3	191.2	177.2
66	v3	8	64	800x600	guided	170.1	170.1	171.8	173.7	176.6	173.2	195.6	196.8	175.3	172.0